Scientific and Large Data Visualization - class notes

by Jingrui Zhu

University of Pisa
a.a. 2025/2026

Contents

Abstract

Disclaimer: These personal notes are intended for educational purposes only and may contain errors or inaccuracies. They are not a substitute for attending lectures, or consulting with lecturers. The content is based on the author's understanding of the course material and may not reflect the official curriculum or the views of the lecturers.

1 Lecture 1 - Introduction

The course is held by external lecturers, Prof.ssa *Daniela Giorgi* and Prof. *Massimiliano Corsini*.

The exam is of 2 parts, a visualization *project* agreed upon with lecturers and an *oral* examination on the notions and concepts presented during the course.

The **main objective** of the course is to learn *how to create an appropriate visualization* for the data we have by learning how to transform data into a graphical representation and when to use different kind of visual representations.

1.1 Introduction

In *scientific visualization*, there is a natural relationship between what is represented and its visual representation, whereas in *information visualization*, the relationship is conventional, and it is up to designer to decide on the best representation.

The goal is to transform data to make sense out ot it.

1.1.1 Information visualization

There are different perspectives to view the information visualization, either as applied science or as a technology.

Information visualization as an **applied science** is that a value of a good visualization let us see patterns in data, and therefore the science of pattern perception can provide a basis for design decisions.

Information visualization as a **technology** is a tool for communication for the understanding and the analysis. The functional constraints from the design of a technological object must depend on the task it should help with, and the graphics form should be constrained by the function of the presentation.

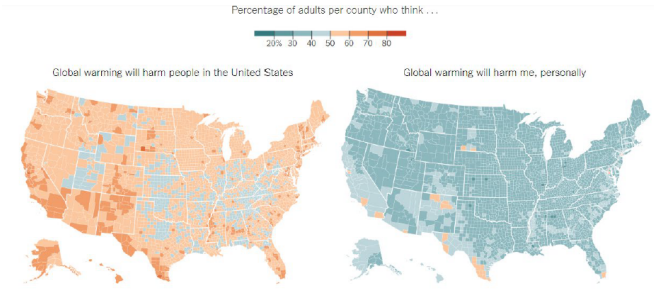
1.1.2 Why information visualization?

Human are a visual species. The human visual system is very good at identifying and analyzing patterns. Visualizations are like cognitive artifacts, the tools that help humans to think better, like an abacus.

Visualization has to do with distributed cognition. Human cognitive system is made of our brain, mind, and sensors, but also of the environment around us, which we use to store and manipulate information.

Visualizations enable parallel processing of information, instead of sequential.

Explanatory visualizations are tools to present information, communicate data and messages, and explain something to somebody else. They are tools for readers to analyze what is being presented to them, and very often, the output of the exploratory analysis generates new questions.



[New York Times, How Americans think about climate change, in six maps]

Figure 1.1: How visualizations convey messages

In confirmatory analysis, the reader has an hypothesis, and uses the visualization to see whether their hypothesis is true or not.

1.1.3 Scientific visualization

Scientific visualization are computing technique for the generation of interactive visual representations of acquired or simulated spatio-temporal data.

Data in ever-increasing sizes makes a graphical approach necessary. The modern process of visualization consists of taking raw data acquired through sensors or produced by some simulation and converting it to a form that is viewable and understandable to humans.

They also help us in identifying patterns and interpreting data, either simulated or acquired data. This can find applications in different fields, engineering, medicine, science, biology,...

2 Lecture 2 - Basics

2.1 Why visualization

Computer-based visualization systems provide visual representations of datasets designed to help people carry on tasks more efficiently.

Visualization allows people to analyze data when they do not know exactly what questions they need to ask in advance. Otherwise, they can use computational techniques from statistics or learning.

If there are many possible questions to ask, the best path forward is an analysis process with a *human in the loop*, with a visual system augmenting human capabilities, rather than replacing it.

Visualization allows people to offload internal cognition and memory usage to the perceptual system, using images as external representations.

Visualization exploits the human visual system as a mean of communication. A significant amount of visual information process occurs in parallel at the preconscious level.

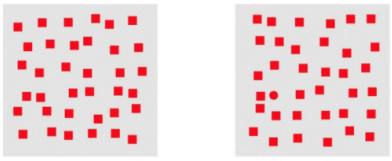


Figure 2.1: Dependence on vision

Human vision can quickly find out the odd one in the fig 2.1, unlike other senses, because of the lack of parallelism in information processing.

2.1.1 Visualization tools

Visualization tools help people in situations where seeing the dataset structure in detail is better than seeing only a brief summary of it.

As the fig 2.2 displays, by transforming numeric data into graphics, we can see how the data values change according to the X and Y.

Interactivity

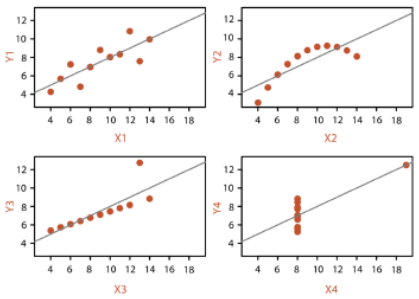
Sometimes, the quantity of data is immense, and the limitations of both people and displays preclude showing everything at once. With interaction, user actions can change the view and investigate at multiple levels of detail. Look at an interactive map service, like GoogleMaps.

1		2		3		4	
X	Y	X	Y	X	Y	X	Y
10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56
7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89

(a) Table summary

	1		2		3		4	
	X	Y	X	Y	X	Y	X	Y
Mean	9.0	7.5	9.0	7.5	9.0	7.5	9.0	7.5
Variance	10.0	3.75	10.0	3.75	10.0	3.75	10.0	3.75
Correlation	0.816		0.816		0.816		0.816	

(b) Table details



(c) Table data in graph

Figure 2.2: Why visualization tools matter

Limitations

Visualizations have limitations, especially when they come from "hardware" limitations. Human limitations are the finite resources for memory and attention. Computational capacity renders scalability a concern. And also, how the information is displayed.

Task focused

The user task is an important constraint on the visualization, as much as the data itself.

Visualization tools can support presentation, discovery, enjoyment of information, or even production of more information for subsequent use. A suitable visualization tool for one task is not necessarily fit for another task on the same dataset (No free lunch). Therefore, we must learn to understand which kind of visual representation to use, depending on both data and task.

Effectiveness

The focus on effectiveness is a corollary of defining visualization to have the goal of supporting user tasks. The goal of the designer is not to make pleasing figures but to render the results effective.

The vast majority of the possibilities in the design space can be ineffective in a specific usage context. This can be caused by poor design match with the properties of the human perceptual and cognitive system, or maybe it is a poor match with the intended task.

2.2 What-Why-How

The What-Why-How method for visualization design is to answer the questions when designing the visualization.

What data is being visualized? *What* data are shown in the views?

Why does the user need a visualization? *Which* is the task being performed?

How is the visual idiom constructed in terms of design choices?

2.2.1 Semantics

The data semantics refers to the real-world meaning of the data. As it should be provided for each dataset, as metadata. For example, the word "Apple" refers to what, the company? The fruit? the city? By means of semantics, we can take care of the ambiguity behind the data.

2.3 Data Type

The type of data is its mathematical or structural interpretation. There are five basic data types in visualization:

- *Items*: an individual entity that are discrete in the dataset. They can be seen as a row in a table.
- *Attributes*: some specific property that can be measured, observed, or logged. In practice, they are properties of objects, also called as variables
- *Links*: a relationship between items, typically within a network.
- *Positions*: 2D or 3D spatial data (latitude and longitude)
- *Grids*: they specify the strategy for sampling continuous data in terms of both geometric and topological relationships between its cells

2.4 Attribute type

The major distinction between attribute types is *categorical vs ordered*.

Categorical attributes are attributes whose values describe categories and do not have an implicit ordering. They can have an imposed external ordering (alphabetical order, for instance), but it is not intrinsic to the attribute itself.

Ordered attributes have an implicit ordering.

One must choose an appropriate visual representation according to the attribute types and semantics

2.4.1 Ordered attributes

Ordered attributes can be further subdivided into:

- **Ordinal**: attributes which have a well-defined ordering but do not support full-pledged arithmetics (shirt size, ranking, etc.)
- **Quantitative**: attributes whose values represent measured quantities/magnitudes and that support arithmetic comparison (temperature, height, price, etc.)

Ordered attributes can have different semantics:

- **Sequential**: there is a homogeneous range from a minimum to a maximum value (height).
- **Diverging**: there are 2 sequences pointing in opposite directions that meet at a common zero-point (temperature has positive/negative value ranges).
- **Cyclic**: the values are wrapped around back to a starting point (hours of a day)

2.5 Dataset types

A dataset is a collection of information that is the target of analysis. The 4 basic dataset types are *tables, networks, fields, and geometry*. Each of them arise from combinations of the data types of items, attributes, links, positions, and grids.

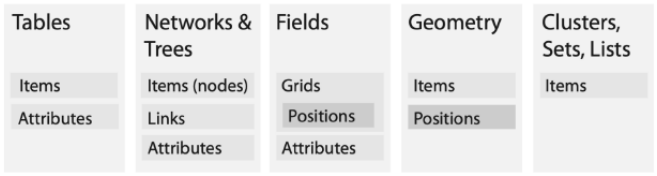


Figure 2.3: Data and Dataset type combinations

Tables

Many datasets come in the form of tables of rows and columns, like in a spreadsheet. In a flat table, each row represents an item, and each column is an attribute of the dataset. Each cell contains a value for that pair. Multi-dimensional tables have slices and multiple keys.

Networks

Networks are suitable to specify that there is some kind of relationship between two or more items. An item in a network is called a node, and a link between 2 nodes is a relationship between two items. A network node and link can have attributes associated with them.

Trees

Trees are networks with a hierarchical structure and no cycles. Each child node has exactly one parent node pointed to it.

A tree is a typical representation for hierarchy, like the organization chart of a company, or an evolutionary relationship between species in the biological tree of life.

Meshes

A 3D mesh is a particular type of graph used in computer graphics to represent 3D models. Nodes are called vertices, arcs are called edges, and there are also polygons called faces.

Geometry

They specify information about the shape of items with explicit spatial positions.

2.5.1 Fields

Fields are a type of dataset in which values associated with a cell contain measurements or calculations from a continuous domain. Since we have a continuous domain, fields require *sampling*, the measurement frequency, and *interpolation*, how to show values in between the sampled points.

Field values are stored in grids, which have 2 properties:

- Geometry: position of cells in space
- Topology: the way cells are connected to adjacent ones

According to the values stored in cells, we can distinguish among scalar fields, vector fields, and tensor fields. These values can be static or time-dependent; deterministic or uncertain.

- *Scalar* fields is one in which each cell contains a single value, for example, gray level values in medical imaging
- *Vector* fields are ones where each cell contains a vector, represented as direction and modules (for instance, velocity)
- *Tensor* fields are the ones where each cell contains a multi-dimensional array of attributes at each point

Continuous data is often found in the form of a spatial field, where the cell structure of the field is based on sampling at spatial positions. The data contain information about both sampled values and the position where they have been acquired.

According to grid geometry and grid topology, we have different types of grids: *uniform, rectilinear, structured, unstructured*.

Uniform grid

They sample at a regular interval; there is no need to explicitly store topology. A uniform grid is formed of tessellations of the Euclidean plane/space by square/cubic elements, points are spaced regularly along each direction.

Rectilinear grid

They support non-uniform sampling for an efficient storage of information, at the cost of having to store geometric locations. They are useful to adapt the sampling to the geometry of the data. Its tessellations are of rectangles/rectangular cuboids, regular grids, but the spacing of points along axes can vary. They are useful to adapt the sampling to the geometry of the data. It has a fixed connectivity, yet needs to store information for spacing.

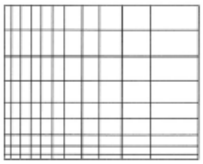


Figure 2.4: Rectilinear grid

Structured grid

They allow a curvilinear shape. It has the same connectivity as the previous two types, but an irregular geometry. Points can be replaced at an arbitrary coordinate, but no overlapping or self-intersections. It has to store the geometry of each cell.

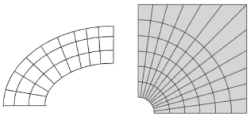


Figure 2.5: Structured grid

Unstructured grid

They have complete flexibility, but need to store spatial positions and topological information about how the cells connect to each other. It allows different combinations of cells (a popular choice).

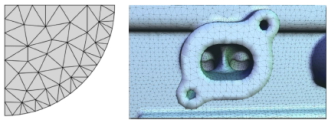


Figure 2.6: Unstructured grid

It can also be under the form of a cloud of points, but this does nor explicit any connectivity information and have irregular geometry.

3 Lecture 3 - Basics

3.1 Visual encoding

Visual encoding means *going from data to visual representations*. Encoding requires not only choosing a graph that is appropriate for the data at hand (its type and semantics), but also selecting the individual graphical elements and their properties.

There are two core graphical elements in a visualization: visual marks and visual channels. *Marks* are basic geometric/graphical elements that depict items or links (a point, a line, a polygon, ...). *Channels* are visual properties that control the appearance of marks (color, dimension, ...) and depict attributes.

They are the building blocks for analyzing visual encoding; even complex visual encoding can be broken down into components that can be analyzed in terms of their marks and channel structure. The effectiveness of marks and channels for encoding data depends on data types.

Definition 3.1 (Marks)**.** *A mark is a basic graphical element, a geometric primitive. They are classified according to the number of spatial dimensions they require:*

- *0-dim: points*
- *1-dim: line, segments*
- *2-dim: areas*
- *3-dim: volumes*

Definition 3.2 (Channels)**.** *Visual channels are a way to control the appearance of marks and encode information about attributes, independent of the dimensionality of marks. Some main channels are:*

- *spatial position: aligned, unaligned, depth, region ...*
- *color: hue, saturation, luminosity*
- *shape and texture*
- *slop and angle*
- *size: length(1D), area(2D), volume(3D)*

Channels can be divided into 2 different categories, according to the two different sensor modalities on the human perceptual system: identity and magnitude channels.

Identity channels give information about what something is or where it is, like *shape, color*.

Magnitude channels tell us how much of something there is, the *length, area, saturation, size, ...*

Definition 3.3 (Context)**.** *A contextual component can make a visualization much easier to interpret, such as legends, labels, annotations, and also grids, axes, reference lines, etc.*

Bar chart Bar chart encodes 2 attributes, a *quantitative* one represented on the y-axis and a *categorical* one represented on the x-axis.

Its typical mark is a line (bars, 1D marks). Two channels, one for vertical spatial position for the quantitative attribute and one for horizontal spatial position for the categorical attribute.

Scatterplot A scatterplot can encode multiple attributes. Its main mark is points (circles, 0D marks). It can have different channels: *one* for vertical spatial position along the y-axis to represent a quantitative attribute; *one* for horizontal spatial position along the x-axis for the second quantitative attribute; *a third channel* could be color to represent a quantitative or qualitative attribute; a forth channel is the area of the points.

3.2 Visual decoding

Visual decoding means deconstructing a visual representation into its major units, and identifying the graphical elements *(what are the visual marks and channels?)* and the mapping rules, i.e., the information that the graphical elements represent *(What data items do the marks represent? What attribute do the channels represent?)*. It is useful for evaluating and redesigning visualizations.

3.3 Expressiveness and Effectiveness

Channels are not equal: the same data attribute encoded with two different visual channels will result in different information content on the users' side, after it has passed through the perceptual and cognitive pathways of the human visual system.

The use of marks and channels in visual design should be guided by the principles of **expressiveness and effectiveness**. There is a ranking of channels according to the type of data that is being visually encoded. Once you have identified the most important pieces of information in your data, you have to ensure they are encoded with the highest-ranked channels.

Definition 3.4 (The Expressiveness Principle)**.** *The visual encoding should express all of, and only, the information that is present in the dataset attributes. Do not represent information and relationships that are not in the data, especially, do not use any color or position when the data does not encode any information.*

Ordered data should be shown so that our visual system perceives them as ordered, by using *magnitude channels.*

Unordered data shouldn't be shown in a way that perceptually implies an ordering that does not exist, use *identity channels*.

Definition 3.5 (The Effectiveness Principle)**.** *The importance of the attribute should match the salience of the channel, i.e., its noticeability. Relevant information should be prioritized, then encoded with the most effective/accurate channels to be most noticeable. Decreasingly important attributes can be matched with less effective channels.*

3.4 Channel rankings

The ranking shows how effective the channels are at conveying different types of attributes.

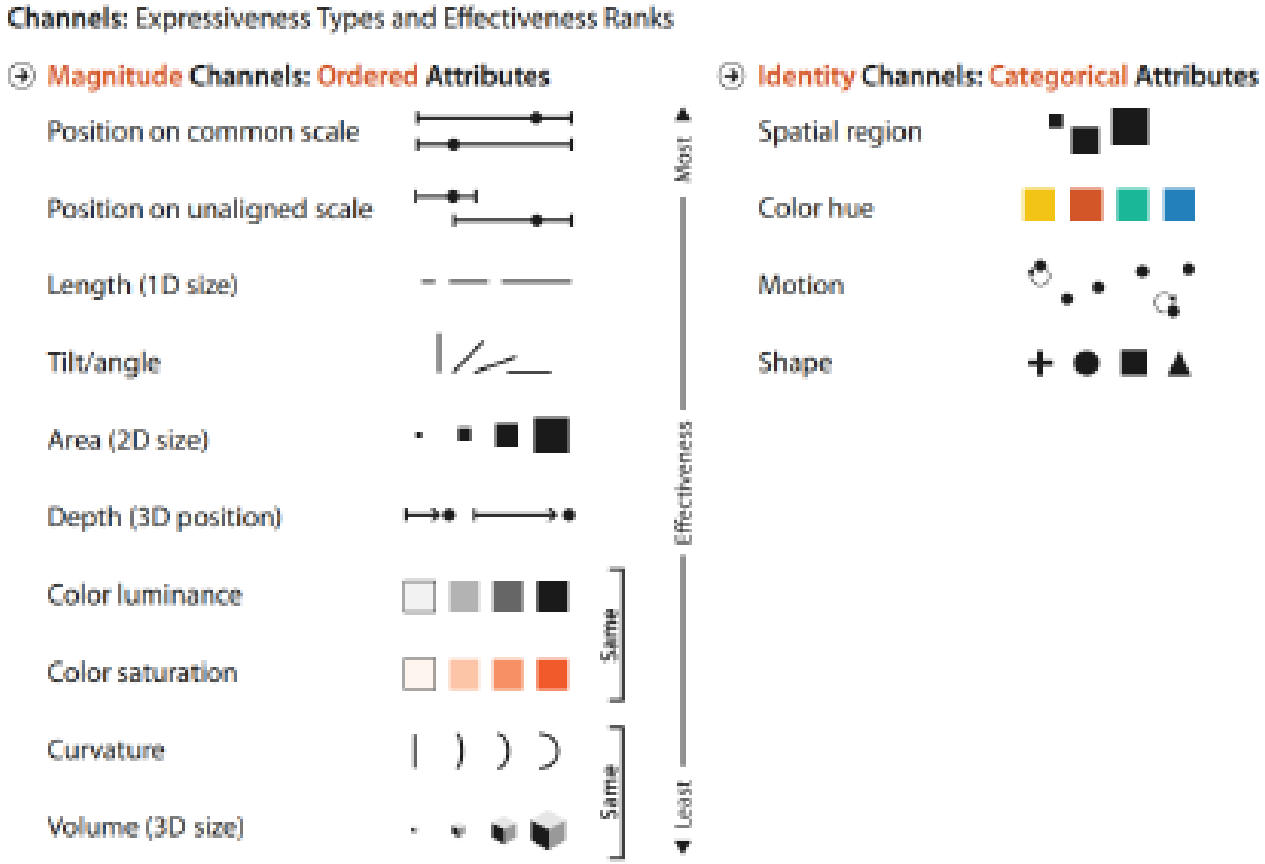


Figure 5.1. The effectiveness of channels that modify the appearance of marks depends on matching the expressiveness of channels with the attributes being encoded.

Figure 3.1: A channel ranking

The fig 3.3 shows one proposed channel ranking. As the figure shows, in both classifications, spatial channels appear at the top of the lists. The choice of which attributes to encode with position is the most central choice in visual encoding, as they will dominate a user's mental model.

3.5 Visual representation, a counterexample

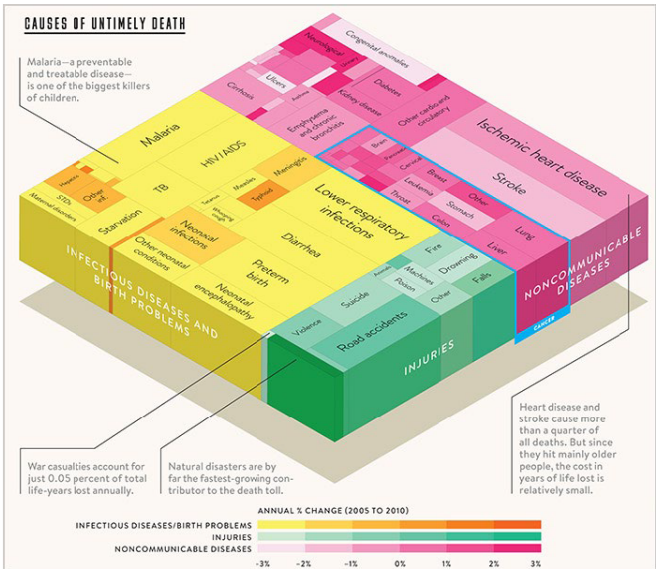


Figure 3.2: A visual counterexample

The fig 3.2 shows a visual representation of causes of death used in a debate. Where, each rectangle represents a death cause, the size of the rectangle represent years of life lost, the color represents the group of death, and the color intensity represents the change from one year to another.

It is a very complicated graph on a very complicated topic; some bad design choices made it less effective than it should be:

- the use of 3D projection is unnecessary, makes the perception of each rectangle much more difficult to comprehend.
- the area is not the best channel to represent quantities, also each rectangle has its own height and width, making the comparisons among causes much more difficult.
- Difficult in evaluating positive and negative changes by the use of color intensity
- Labels are too small to read

3.6 Channel effectiveness

One would naturally ask how channel rankings are justified, and why some channels are better than others. Can all channels be used independently, or do they interfere with each other? We need a scientific answer.

3.6.1 Accuracy

$$S = I^n$$

S = perceived sensation; I = physical intensity; n = exponent.

The power law of Stevens models the response to sensory experience. Its exponent n ranged from the sublinear 0.5 for brightness to the superlinear 3.5 for electric current.

The sublinear phenomena are compressed, superlinear phenomena are magnified. Doubling the physical brightness results in a perception that is considerably less than twice as bright.

Humans can only perceive length as a very close match to the true value; all other visual channels are not perceived as accurately.

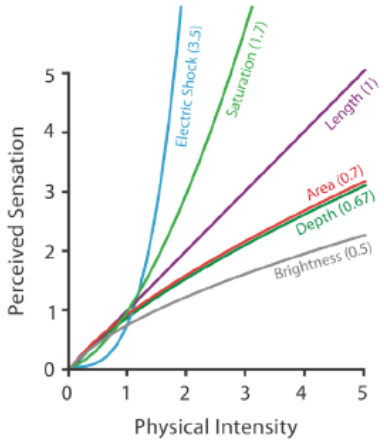


Figure 3.3: The power law of Stevens

3.6.2 Discriminability

Match the ranges: the number of different values that one needs to show for the attribute being encoded must not be greater than the number of bins available for the visual channel used to encode it. IF they do not match, one should use a different channel or aggregate the attributes into meaningful bins (highlights the most important one and aggregate all others under the same value, as others).

3.6.3 Separability

Channels may have dependencies and interactions with others,

- from orthogonal and independent separable channels to the combined integral channels.
- Position and hue are completely separable.
- Size is not fully separable from color hue.
- Horizontal and Vertical size are an integral pair.

3.6.4 Pop-out

Many visual channels provide visual pop-outs, where a distinct item stands out from many others immediately. It has an added value if the time it takes us to spot a different object does not depend on the number of distracting objects. As a general rule, *use pop-out for a single channel at a time*. Most

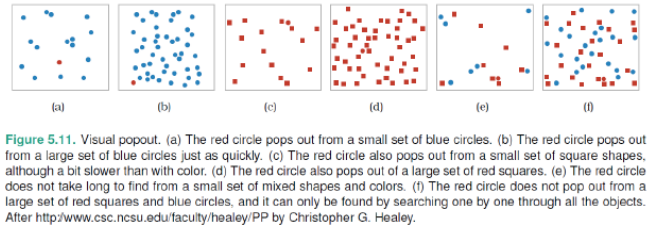


Figure 5.11. Visual popout. (a) The red circle pops out from a small set of blue circles. (b) The red circle pops out from a large set of blue circles just as quickly. (c) The red circle also pops out from a small set of square shapes, although a bit slower than with color. (d) The red circle also pops out of a large set of red squares. (e) The red circle does not take long to find from a small set of mixed shapes and colors. (f) The red circle does not pop out from a large set of red squares and blue circles, and it can only be found by searching one by one through all the objects. After http://www.csc.ncsu.edu/faculty/healey/PP by Christopher G. Healey.

Figure 3.4: Visual pop-out

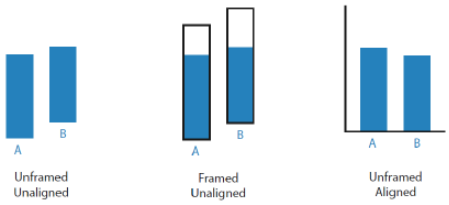
pairs of channels do not support pop-out (except for the shape and color pair). Pop-out is not possible with 2 or more channels

3.7 Relative Vs Absolute Judgments

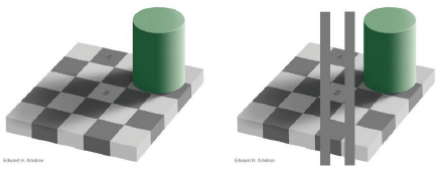
The human perceptual system is based on relative judgments, not absolute ones. The length difference we can detect is a percentage of the object's length.

When two objects are directly next to each other and aligned, we can make a much more precise judgment than when they are not.

Our perception of color and luminance is completely contextual.



(a) Are the 2 objects the same height?



(b) Are there different shades of gray?

Figure 3.5: Relative judgment examples

4 Lecture 4 - Applied Perception

As we have already seen, information visualization is about transforming data into visual representation so that a human can extract useful information out of it. The effectiveness of a visual representation is not arbitrary; it strongly depends on how brain works. That's why understanding how perception works can help one make informed decisions about visualization designs.

Definition 4.1 (Contrast Sensitivity Function)**.** *Our perception is sensitive to pattern contrast frequency and orientation, also color influences the perception as well.*

Definition 4.2 (Perceptive field in retina)**.** *The perceptive field of a cell is the visual area over which a cell responds to light. Retinal ganglion cells are organized with circular receptive fields, which are stimulated on-center when excited and stimulated off-center when inhibited.*

4.1 Grayscale maps

The visual effects can result in large errors when reading quantitative information maps displayed using a grayscale map –> use grayscale maps to represent a few values, and not to map or to compare many values.

It is useful for highlighting or show boundaries, important items by reducing contrast of unimportant ones; or to adjust background luminance to obtain better readability.

4.1.1 Cornsweet illusion

The lateral inhibition can be considered part of an edge detection process in a scene under viewing. Pseudo-edges can be seen depending on the stimulus.

The brain does perceptual interpolation so that regions affected by such edges can appear lighter or darker. This is called *Cornsweet illusion*.

Figure 4.1: The Cornsweet effect

In the fig 4.1, an example of the Cornsweet effect shows the separating line between the shades of gray.

4.2 Eye movement

- *Saccadic* movement: *ballistic* movements of the eyes that change the point of fixation. They can be voluntary or stimulus-elicited
- *Smooth-pursuit* movement: *slow tracking* movements of the eyes to keep a moving stimulus on the fovea (a nerve in the eye)
- *Vergence* movement: align the fovea of each eye to a target according to its distance
- *Vestibulo-ocular* movement: *stabilize* the eyes compensating for head movements

4.2.1 Saccadic movement and fixations

The Saccadic eye movements take between 20 to 180 ms. It makes both eyes move in the same direction. This movement may not be a simple linear trajectory.

Definition 4.3 (Fixation)**.** *A fixation is composed of slower and finer movements (microsaccades, tremors, and drift) that help the eye align with the target. A fixation varies between 50 and 600 ms.*

For instance, a typical movement of this kind during reading moves the eyes for 2 degrees, and in general, the movement is between 2 and 5 degrees. If the movement is larger than 20 degrees, the head movement is required.

4.3 Pre-attentive process

Some visual stimuli naturally "pop up" from their surroundings, especially when all the items are of the same characteristics except for one. These "pop-up" features can be: orientation, curvature, shape, size, color, light/dark, enclosure, concavity/convexity, or addition.

(a) orientation (b) Light/Dark (c) Length

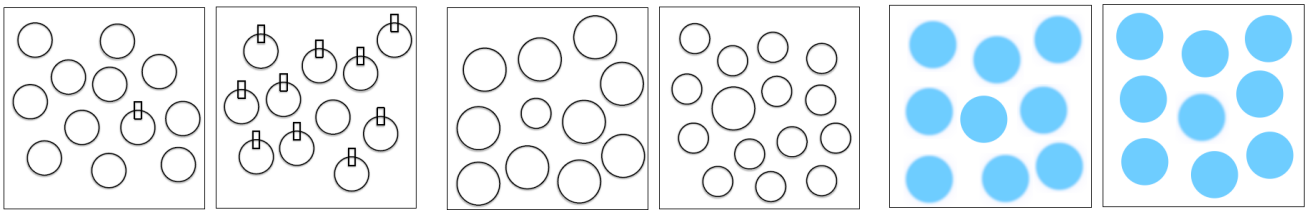
Figure 4.2: Features that pop up

In the fig 4.2, some examples of how some visual features are pre-attentively processed.

4.3.1 Asymmetry

Some pre-attentive processes are not symmetric.

- Adding marks is more efficient than removing marks
- Increasing sharpness is more efficient than decreasing sharpness
- A big object surrounded by small objects is more efficient than the other way around



(a) Marks (b) Size (c) Sharpness

Figure 4.3: Asymmetry of pre-attentive process

In the fig 4.3, some examples of how some of the asymmetry of the pre-attentive process works. For instance, an object with an additional mark is much more noticeable than an object without a mark.

4.3.2 Feature combination

Note that different combinations of pre-attentive visual features may not be pre-attentive.

In the fig 4.4 shows an example of combining shape and color visual features in an attempt to obtain the "pop up" effect, but the actual result is not as good as we thought.

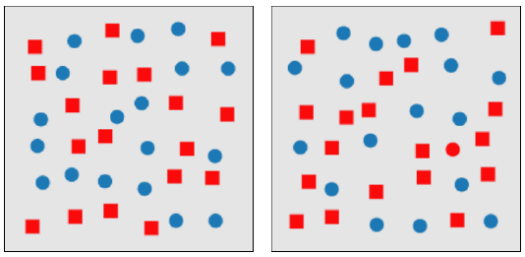


Figure 4.4: Where is the red circle?

4.4 Gestalt laws

It is an attempt to understand pattern perception. It is a powerful ally in visual perception, for example, to show group elements, to show relationships, to make pattern comparisons, in an effective way:

- Proximity: objects close to each other are perceived to form a group
- Similarity: similar objects are perceived to form a group
- Connectedness: connected objects are perceived as related; most of the time, connecting different objects with a line is a powerful way to express a relationship between the objects.
- Continuity: we expect that a line or an edge continues to follow its direction and does not deviate from it
- Symmetry: objects arranged symmetrically are perceived as forming a visual whole instead of being perceived as separate entities
- Closure: we tend to perceive the complete appearance of an object so that our brain fills the gap in case of missing parts
- Common fate: we tend to perceive a group of object that moves in the same direction as a whole
- Figure-ground: the perceptual effect regards the formation of a figure from the background

(a) Connectedness (b) Closure (c) Figure-ground

Figure 4.5: Gestal laws examples

5 Lecture 5 - Applied Perception II

Color is extremely useful in data visualization to show patterns, for labeling, and for highlighting. However, color is easy to misuse. Think of "color" as an attribute of an object rather than its primary characteristic.

5.1 Basic color perception

Human have three distinct color receptors on the retina, called the *cones*, which are active at normal light levels. With the support of rods on color perception, we can perceive light or a "shadowy figure" in low luminosity conditions.

Definition 5.1 (Trichromacy theory)**.** *Cones are sensitive to different wavelengths(short, medium, long), hence, they absorb light around the spectrum of* blue, green, and red*. The theory says that we perceive color as a three-channel system.*

All color spaces, even if designed to different purposes, as three-dimensional.

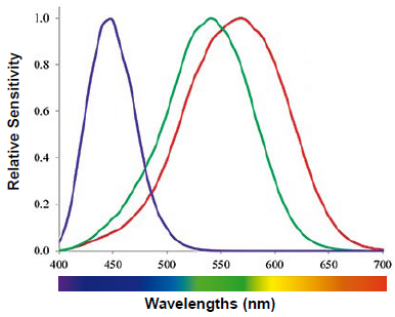
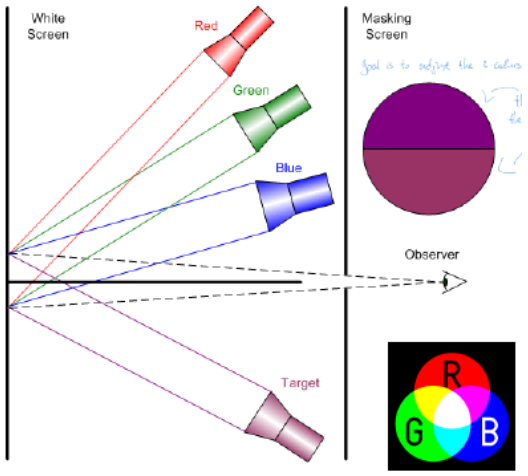


Figure 5.1: Thrichromacy theory

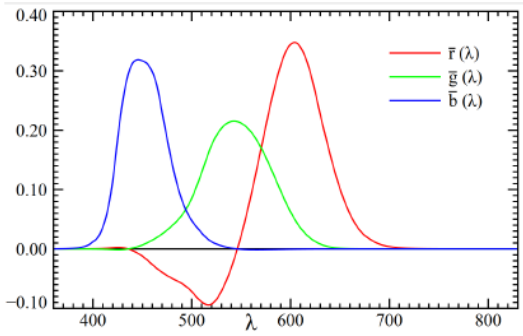
5.1.1 Color measurement and specification

Since only three different receptors are involved in color vision, it is possible to match a patch of color light using a mixture of three color lights, called *primaries*. The color experiment consists in using a transformation to create the same color on different output devices.

In fig 5.2a we show the experiment setup and results, the goal of the experiment is represented in "making screen", adjust the colors so that the two halves of the circle show the same color.



(a) Experiment setup



(b) Experiment results as functions

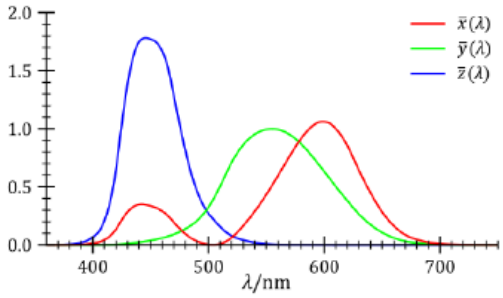
Figure 5.2: Color measurement experiment

In fig 5.2b we show the experiment result on how sensitive our eyes perceive the primaries, the so-called **RGB color space**, created by the CIE - International Commission of Color.

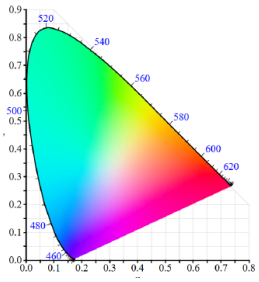
The RGB color matching function can also be represented as a *function of X, Y, and Z*, where:

- Y corresponds to the perceived luminance in well-lit condition
- Z is close to the short cone response
- X is a mix such that the values become non-negative

And for a fixed value of Y, the plane XZ contains all the possible chromaticities at that luminance.



(a) Color matching function as XYZ



(b) Chromaticity diagram

Figure 5.3: XYZ color space

Definition 5.2 (Gamut)**.** *Gamut is the set of color spaces that a device, such as a printer or a monitor can reproduce. Most of the time, it is a subspace of the chromaticity diagram*

5.1.2 Opponent process theory

The theory was proposed by Hering, a German psychologist, and later supported by experiments. Hering proposed that cones combine their stimulus, forming three pairs of colors that compete to form the final one. These pairs, called *opponent pairs* are *black-white, red-green, and yellow-blue*.

This theory suggests that the visual system interprets color by processing signals from three types of cone cells in an antagonistic manner, meaning that the activation of one color inhibits the perception of its opposite.

One interesting implication is the *afterimages*. For example, if you stare at a bright color (like red) for a long period of time and then look at a white-colored surface, you may see a cyan afterimage. This happens because the red receptors become fatigued, leading to the perception of their complementary color.

It also suggests the extremes of the opponent channels, which are fundamental to our understanding of color perception. These unique hues are *red, green, blue* which cannot be created by mixing other colors.

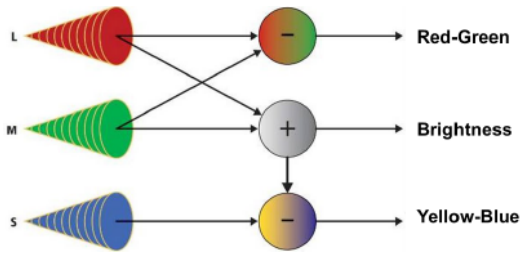


Figure 5.4: Opponent process theory

In fig 5.5 shows an example of this theory in practice. When looking at this picture for at least 30 seconds, and focus at the little white dot in the middle. Then switch to a white colored surface, and what we see on the white background is the flag of the USA. This is because when you stare at these colors, you are exciting the same source of these colors, and when you switch to the white background, since these sensors have been excited for too long, they inhibit those colors as the only colors.

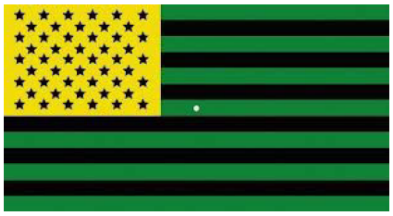


Figure 5.5: Opponent process theory, an example

5.2 Color spaces

Saturation means how pure the color is, intended as its brightness.

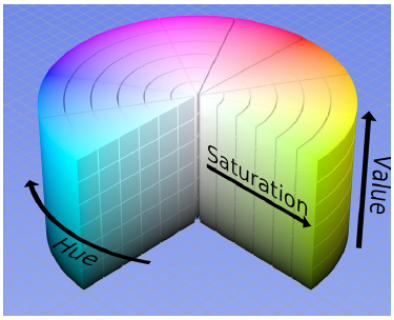


Figure 5.6: Hue Saturation Value

Color specification can be represented as in fig 5.6, which is a more natural representation than with RGB. But, they are not perceptually uniform spaces (calculated Euclidean distance in the color space does not correspond to perceptual distances).

Having a color space in which equal perceptual distances are useful to specify color tolerances, color codes, pseudo-coloring using sequences of colors to represent data values, possibly with perceptually equal steps.

The Euclidean distance is meaningful from a perceptual point of view for the color space. It is defined in eq 5.1.

$$\Delta E^*_{ab} = \sqrt{(L^*_2 - L^*_1)^2 + (a^*_2 - a^*_1)^2 + (b^*_2 - b^*_1)^2} \tag{5.1}$$

If the result $\Delta E < 1$, colors are not perceptible; for $1 < \Delta E < 2$, close observation needed to perceive the difference; for $2 < \Delta E < 10$, the colors are similar but different. It is important to know that ΔE is not perceptually uniform as originally intended and therefore suspended by specifications.

Although uniform color spaces only provide a rough first approximation of how color differences will be perceived, an important influencing factor is size: we are much more sensitive to differences between large patches.

Tips: Use saturated colors when coding small symbols or thin lines, and less saturated colors for large areas, as showed in fig 5.7.

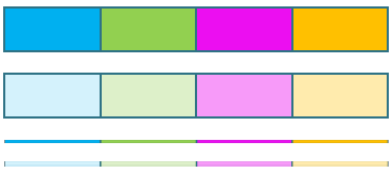
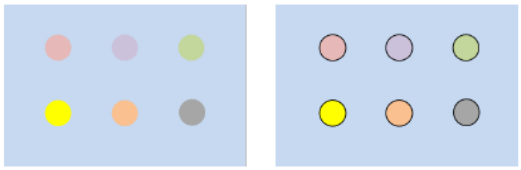


Figure 5.7: Color differences and use of saturation

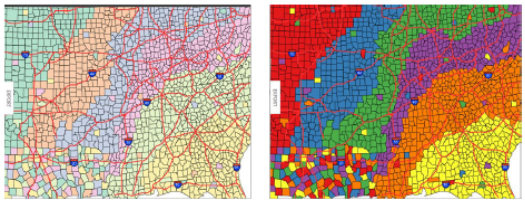
5.3 Color and Visualization

The red-green and yellow-blue chromatic channels presented from the opponent process theory are each capable of carrying only about 1/3 of the amount of detail carried by the black-white channel.

Purely chromatic differences are not enough to display fine details, ensuring adequate luminance contrast with the background, and also colors with different chromaticity (saturation) can be used to improve readability.



(a) Contrast boundary to improve readability



(b) Less saturated colors for large patches and saturated colors for finer details

Figure 5.8: Visualization and color properties

In 1986, Post and Greene carried out an experiment on the naming of colors; among 210 different colors, only 8, plus white, were consistently named. This result suggests that only a few colors can be used as categorical labels. Nowadays, the 12 recommended colors shown in fig 5.9 are widely agreed-upon category names with reasonably far apart in color space. These colors are: red, yellow, black, pink, gray, brown, green, blue, white, cyan, orange, purple.

Figure 5.9: Color labels

Color and Semantics

There are no universal conventions on associating a color with some semantics, but they are widely used. For instance, red for hot/bad/danger, blue for cold, green for life/go, etc. The semantic association with gray is that of belonging to an unspecified category, which can be used for highlighting more important information using different colors.

Color ramps

Color ramps are a predefined set of colors used to present a theme (green-yellow hue of colors represents summer, red-orange-yellow hue of colors represents autumn ...).

There are some guidelines in using color ramps, for instance, avoid color ramps problematic for color-blind individuals; use a spectrum-based color ramp when its use is deeply embedded in the user culture; to reveal fine details, use a pseudo-color sequence that varies in luminance, not only in chromaticity.

6 Lecture 6 - Tables

Definition 6.1 (Key and Value)**.** *A* key *is an independent attribute that can be used as a unique index to look up items in a table, while a* value *is a dependent attribute.*

When deciding on the visualization, one should know the number of keys and the number of values in the table, so that different types of tables can be more suitable.

6.1 Scatterplot

Scatterplot encodes two quantitative attributes (values) using vertical and horizontal spatial position channels, and the mark type is point. It is effective for providing overviews and characterizing distributions, specifically for finding outliers and extreme values. It can also be used to judge the correlation between 2 attributes.

Correlation

With Visual encoding, the task corresponds to the easy perceptual judgment of noticing whether the points form a line along the diagonal. The stronger the correlation, the closer the points fall along a perfect diagonal line; positive correlation is an upward slope, and negative correlation is a downward slope. In addition, a regression line is often superimposed on the raw scatterplot to show the best fit between attributes. One can even use a transformation to derive a clearer relationship between attributes. It is suited for up to hundreds of points (items) for the scalability issue.

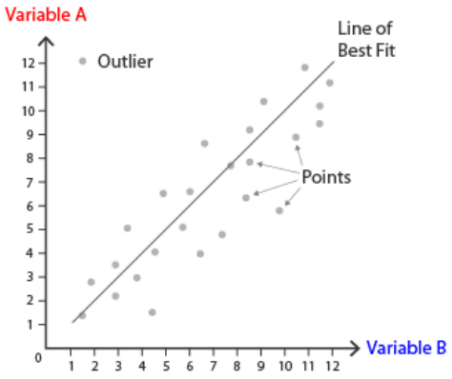


Figure 6.1: Scatterplot for feature correlation

Recap: Scatterplot

Data (What)	Table with 2 quantitative value attributes
Encode (How)	Express values with horizontal and vertical spatial position and point marks
Task (Why)	Find trends, outliers, distribution, correlation; locate clusters
Scale	hundreds of items

6.2 Bar chart

A bar chart uses a line mark and usually encodes a categorical or ordinal key attribute using position along the horizontal axis as a channel and a quantitative value attribute with position along the vertical axis. Each bar is in a separate region of space, and there is one for each level of the categorical attribute.

It is useful to visualize how a measured quantity is distributed across categories, and also for looking up individual values. Bars are all aligned within a common frame, so that the highest-accuracy aligned-position channel is used.

Bars can be ordered alphabetically by the categories, which makes look-up by name easy, but often hinders the meaningful patterns in the dataset. Another ordering is by the values of the quantitative attribute that is encoded by the bar heights; this data-driven ordering makes it easier to see dataset trends.

The scalability is a big issue here; to accommodate bars, sufficiently large white space should be allocated so that each bar line mark is distinguishable. *Less is more...*

Percentage stacked bar chart

Unlike a simple stacked bar chart, which uses the absolute values. Percentage bar chart uses the normalized values so that the sum of segments in a single bar is 100%. This can be used to better show the relative difference between quantities in the different groups.

Recap: Bar charts

Data (What)	Table with one quantitative value attribute and one categorical key attribute
Encode (How)	Line marks, express value attribute with aligned vertical position, separate key attribute with horizontal position
Task (Why)	Lookup and compare values
Scale	up to hundreds of levels of key attributes

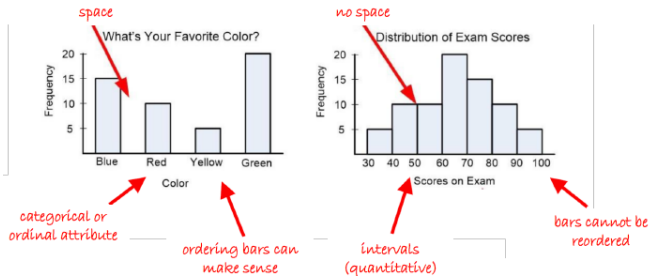


Figure 6.2: Difference between a bar chart and a histogram

6.3 Histogram

A histogram visualizes the distribution of values of a quantitative attribute. It uses the same marks and channels as bar charts, but a histogram usually shows derived or aggregated data; there is no space between bars; it shows the continuous distribution of the data, and thus makes no sense to order bars.

Recap: Histogram

Data (What)	Table with one quantitative value attribute
Derived (What)	Derived table is the one where one derived ordered key attribute (bin) and one derived quantitative value attribute (item count per bin)
Encode (How)	Rectilinear layout, line mark with aligned position to express derived value attribute. Position is judged by key attribute

6.4 Line chart

A line chart visualizes an ordered key attribute using horizontal position as a channel and a quantitative value attribute using vertical position as a channel.

The marks are points and line segments connecting consecutive point pairs. Segments are connection marks to emphasize the ordering of the items along the key ais by explicitly showing the relationship between one item and the next. Thus, they have a stronger implication of trend relationships, making them suitable for the task of spotting trends. Its popular applications include finance, climate studies, public policies ...

Area Chart

It is a variation of line charts with the area below the line filled in.

Recap: Line charts

Data (What)	Table with one quantitative value attribute and one ordered key attribute
Encode (How)	Dot chart with connection marks between dots
Task (Why)	Show trend
Scale	up to hundreds of levels of key attributes

6.5 Multiple attributes

6.5.1 Bubble Chart

It is a variation on a scatterplot, accommodated to represent additional attributes, where circle areas and colors can represent additional attributes. One could add even more channels and attributes, but it would also make information decoding much harder. Too many bubbles can make the chart hard to read; therefore, bubble charts have a limited data size capacity. However, this can be solved through interactivity, by, for example, a filter for categories, clicking over a bubble to display hidden information, and so on.

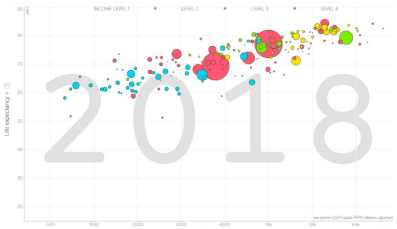


Figure 6.3: Bubble chart

6.5.2 Stacked Bar chart

It is a more complex glyph for each bar, with multiple sub-bars stacked vertically. It has two key attributes that can have as many composite bars as the number of values for the first key attribute and as many sub-components within each bar as the values for the other key attribute. Both the length of the composite bar and the length of each sub-component encode values, resulting in a multidimensional table.

Composite glyphs are arranged horizontally according to the primary key; the secondary key is used to construct the vertical structure of the glyph, and color is often used alongside length coding.

In fig 6.4 shows an example of a stacked bar chart:

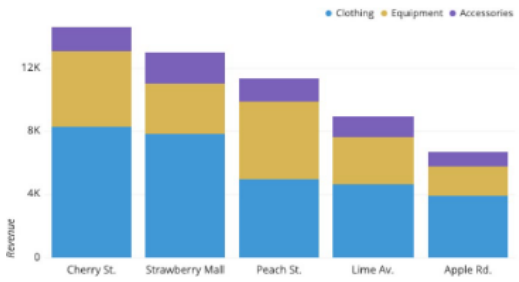


Figure 6.4: Stacked bar chart

- the full bar height (representing the first key attribute) shows the value for the combination of all items in the stack;
- the height of the full combined bar and the lowest sub-component is easy to read and to compare against each other because they have an aligned starting point
- each sub-component is represented with a different color, which represents the second key attribute, it has as many values as the number of sub-components (also color, in this case)
- The ordering of stacking is significant, as it determines the patterns that are most easily visible

Recap: Stacked Bar charts

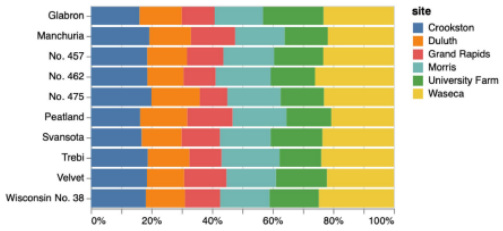
Data (What)	Multidimensional table with one quantitative value attribute and two categorical key attributes
Encode (How)	Bar glyph with length-coded sub-components of value attribute for each category of secondary attribute. Separate bars by category of primary key attribute
Task (Why)	Part-to-whole relationship, lookup values, find trends
Scale	up to hundreds of levels for the main axis key attributes. Several levels for the secondary (stacked glyph axis) key attribute

Normalized Stacked Bar chart

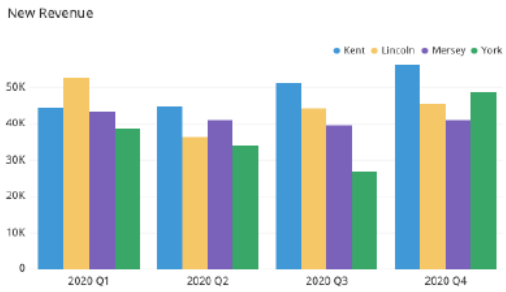
It is a variant of stacked bar chart in which the quantitative attribute is normalized, showing proportion instead of absolute values

Grouped Bar chart

It is another representation of a stacked bar chart, with two key attributes. The same bar chart is repeated multiple times for the number of categories in the second attribute. It is an excellent tool for the comparison of individual values



(a) Normalized Stacked Bar chart



(b) Grouped Bar chart

Figure 6.5: Bar chart variations

6.5.3 Line chart variations

Stacked Line chart

A variation of a line chart in which values across categories are stacked one on top of each other. It has a big noticeable limitation: it is not good for reading and comparing individual values over time, since the values are stacked, and the shape of lines is affected by the shape of lines below it. This can be somehow mediated through a filter interaction. It can be meaningless for data that shouldn't be summed, like temperature.

Line chart Series

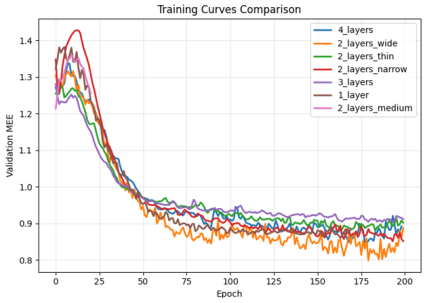
It is a collection of lines in the same chart, easy to use, and has high readability, but to be careful to not to overfill the chart with lines.

Small multiples

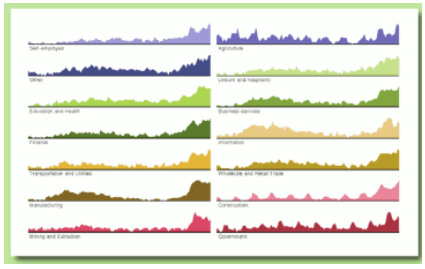
An alternative way to show multiple attributes that hold for all types of charts by creating multiple replicas of the same graph, with each graph in its own chart. It is sometimes more effective than a single plot including too much data.

Flow maps

Depict the movement of entities in space and time by placing stroked lines on top of maps. A flow map encode a large amount of multivariate information (thickness, color, etc. can encode additional information)



(a) Line chart series



(b) Small multiples

(c) Flow map

Figure 6.6: Line chart variations

6.5.4 Heatmap

A heatmap is a 2-dimensional matrix alignment to arrange two key attributes and one value attribute. One key is distributed along the rows and the other along the columns; each rectangular cell in the matrix is occupied by an area mark encoding a single quantitative value attribute with color.

The use of a heatmap allows us to visually encoding quantitative data with color using small area marks, making it very compact. It is good for overviews with high information density and is highly scalable. Keys can also be ordered to identify patterns, even if it is a categorical attribute (can be ordered according to color).

But, its use discriminates a large number of distinct values; only a small number of different levels of the quantitative attribute can be distinguishable, because of the limits of color perception in small, non-contiguous regions.

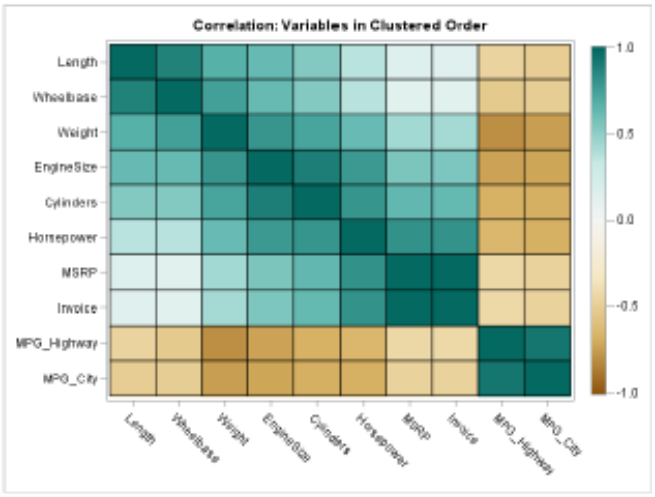


Figure 6.7: Heatmap

Recap: Heatmap

Data (What)	Table with one quantitative value attribute and two categorical key attributes
Encode (How)	A 2D matrix alignment of area marks, diverging color map
Task (Why)	Find clusters, outliers, summarize
Scale	Can have up to one million items; hundreds of levels of categorical attributes; tens of levels of quantitative attributes

6.6 Other visualization tables

Wordcloud

It is a representation of how frequently words appear in a given body of text through the size of the word. It can have various variations in arrangement and color. Mainly used for aesthetic reasons.

Calligrams

It is an image created by arranging the texts to form a thematically related image

Multidimensional icons

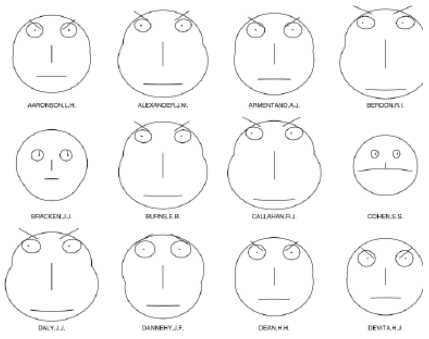
The graph that encode properties of an object in a simple iconic representation.

Petals as glyph

The graph visualize a life index, it maps several variables related to material living conditions and quality of life into petals of different size, to compare well-being across countries.

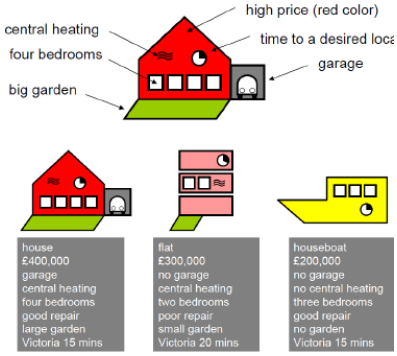
Chernoff faces

It is introduces based on the assumption that human are good at recognizing faces. Variables are mapped to facial features (width/curvature of mouth, vertical size of face, size/slant/separation of eyes, size of eyebrows...). It needs a legend. *It is not a pre-attentive process...?*

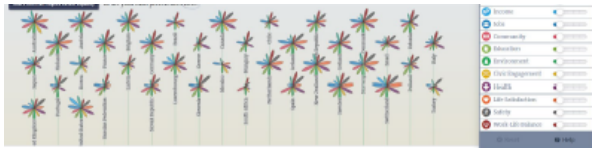


(a) Chernoff faces

(b) Calligram



(c) Multidimensional icons



(d) Petal as glyph

(e) Wordcloud

Figure 6.8: Other visualization tables

7 Lecture 7 - Charts

Spatial axes are an important ingredient in visualization. One should always label the axes and set the standard unit of distance between two points in the chart. This layout can be:

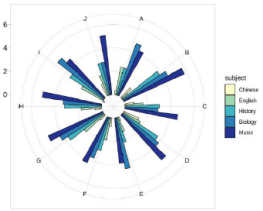
- rectilinear: items distributed along two perpendicular axes, with values ranging from a minimum to a maximum. (like in most of the tables mentioned in Chapter 6)
- parallel: axes placed parallel to each other (in the case of a slope chart)
- radial: items distributed around a circle, using the angle channel (like in a pie chart)

Radial layout

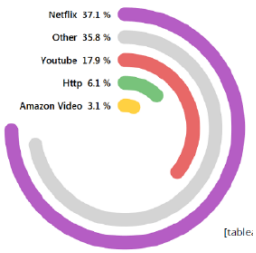
Items in a radial layout are distributed around a circle, using the angle channel. The most natural coordinate system in radial layout is polar coordinates, where one dimension is measured as an angle from a starting line, and the other dimension is measured as a distance from a center point. Compared to a rectilinear spatial position channel, the angle channel is less accurately perceived.

7.1 Radial bar chart

The radial bar chart is similar to a bar chart, using the same line marks to encode a quantitative attribute with the length channel, the only difference being the radial vs rectilinear orientation of the axes. This chart can be messy to the eye; it is usually a good idea to re-order the bars to that the comparison from the highest to the smallest values can be perceived. This chart can be revisited to encode multiple attributes.



(a) encoding multiple attributes



(b) different forms of radial bar chart

Figure 7.1: Different use of radial bar chart

Recap: Radial bar chart

Data (What)	Table with one quantitative value attribute and one categorical key attribute
Encode (How)	Length coding of line marks, radial layout

7.2 Pie chart

The pie chart is the most commonly used radial graphic; it uses 2D area marks and the angle (arc length) channel. It is used to show percentages and proportions among categories by dividing a circle into proportional segments, with each angle/arc length representing the value/proportion for each category, and the full circle represents the sum of all data, i.e., 100%.

Despite the popularity, pie charts are clearly problematic when considered according to the visual channel properties. While it is useful for giving a quick idea and to compare one slice against the total, it is not good for an accurate comparison: angle judgments on area marks are less accurate than length judgments on line marks; the wedges vary in width along the radial axis, from narrow near the center to wide near the end, making the area judgments difficult. In addition, scalability is an issue; it can shows up to tens of categories, simply because as the number of categories increases, the size of each segment decreases, not to mention the size of the legend would also become proportionally large.

The most useful property of pie charts is that they show the relative contribution of parts to a whole; however, this property is unique to pie charts. A single bar in a normalized stacked bar chart would do the same job with a more accurate channel of length.

A pie chart is also one of the most commonly *misused* visualization:

- sum of the percentage of all slices differ from 100%
- redundant representations of data (sometimes people would put a number, a percentage of the slice to the whole, together with the slice)
- unnecessary and unjustified channel, like 3D (applies to all types to visualization)
- incomplete pie chart showing only some slices, *but*, depending on the context, it can be useful for emphasizing the portion on the given topic, thus, showing comparison between different subjects.

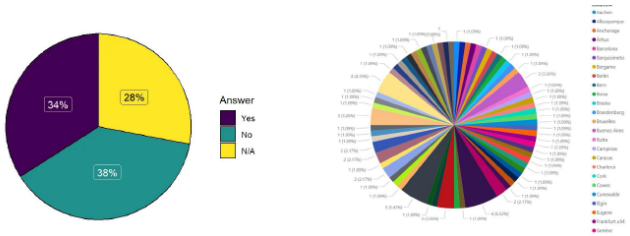


Figure 7.2: Pie chart misuses

Donut chart

It is a variant of a pie chart with the center area cut out, making the focus on arc length rather than on area. It is more efficient on space occupancy while providing more space for labels.

Polar area chart

Another variation of a pie chart that varies the length of the wedge, rather than varying angles.

Radar chart

Also called a spider chart or star chart, where items are displayed along circles, it is useful for periodic data. This chart can easily compare properties of a class of objects or a category, but not as much in understanding the trade-off between variables. One should be careful when using it for many variables or many data points.

Sunburst chart

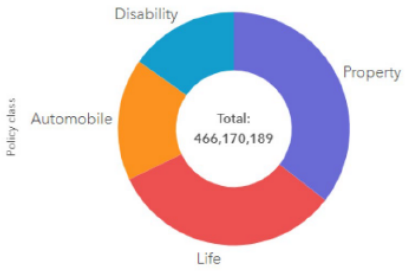
It has a radial layout but can be used to visualize hierarchies. A sunburst chart visualizes hierarchies (a tree structure) with concentric circles where each ring represents a level in the hierarchy, moving outwards.

It used multiple channels, such as distances from the center representing levels in the hierarchy, the angle representing connections among items (nodes), and color can also give additional information about the items. Note that each ring does not have to be complete, as a tree does not have to be perfectly balanced.

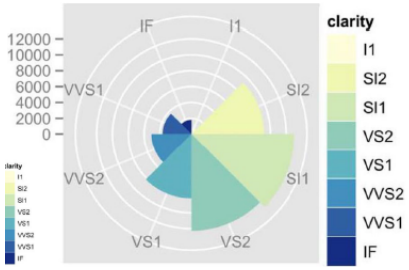
7.3 Treemap

A treemap is an alternative way to visualize hierarchies, by displaying quantities via area size. Each rectangle represents a category, with nested subcategory rectangles. Child nodes are contained in the area region representing the parent node. When a quantity is assigned to a category, its area size is displayed in proportion to that quantity and to the other quantities within the same parent category in a part-to-whole relationship.

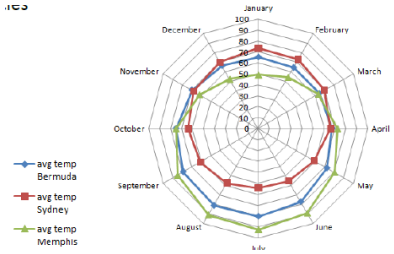
The treemap is very compact and space-efficient, but it is not as good as a sunburst chart in showing the levels in the hierarchy, because only leaf attributes are displayed. In general, it is a good option for showing the overall structure and providing an estimated comparison via area sizes.



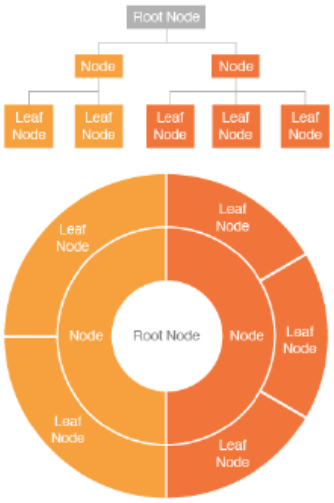
(a) Donut chart



(b) Polar area chart



(c) Radar chart (or spider chart)

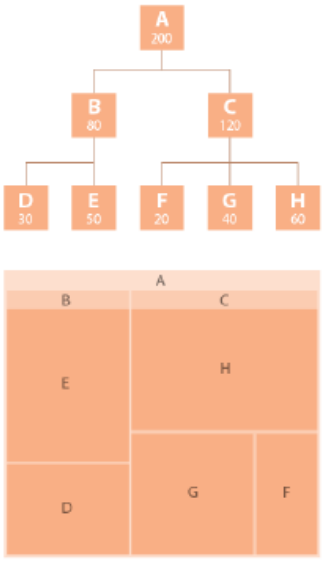


(d) Sunburst chart

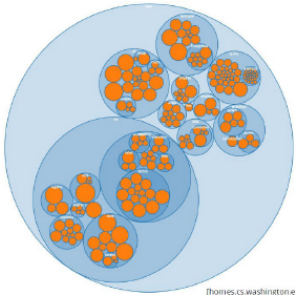
Figure 7.3: Variations of a pie chart

Bubble hierarchy

As with a treemap, it uses containment to represent hierarchies and area to visualize quantities, but with bubbles-shaped containers rather than rectangles.



(a) Treemap



(b) Bubble hierarchy

Figure 7.4: Treemap and Bubble hierarchy variant

7.3.1 Different representations of a tree dataset

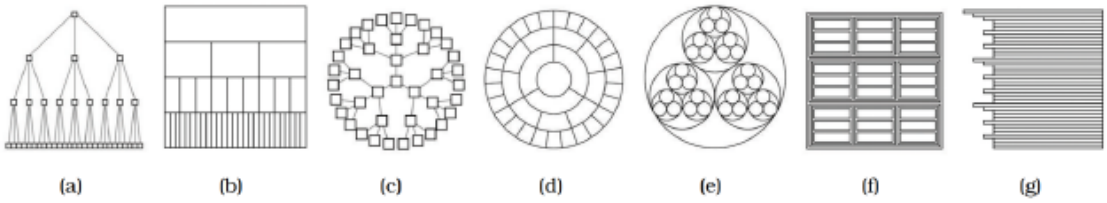


Figure 9.9. Seven visual encoding idioms showing the same tree dataset, using different combinations of visual channels. (a) Rectilinear vertical node–link, using connection to show link relationships, with vertical spatial position showing tree depth and horizontal spatial position showing sibling order. (b) Icicle, with vertical spatial position and size showing tree depth, and horizontal spatial position showing link relationships and sibling order. (c) Radial node–link, using connection to show link relationships, with radial depth spatial position showing tree depth and radial angular position showing sibling order. (d) Concentric circles, with radial depth spatial position and size showing tree depth and radial angular spatial position showing link relationships and sibling order. (e) Nested circles, using radial containment, with nesting level and size showing tree depth. (f) Treemap, using rectilinear containment, with nesting level and size showing tree depth. (g) Indented outline, with horizontal spatial position showing tree depth and link relationships and vertical spatial position showing sibling order. From [McGuffin and Robert 10, Figure 1].

Figure 7.5: Visualizing a tree dataset

8 Lecture 8 - Geospatial data and Best Practice

Recap *Geometry* dataset type

The geometry dataset type specifies information about the shape of items with explicit spatial positions. The items can be represented as points, lines, or surfaces. This type of data may not have any attributes.

The given spatial position is an attribute of primary importance; the central tasks often revolve around understanding spatial relationships. A sensible visual encoding choice is to use the provided spatial position as the substrate for the visual layout, rather than to visually encode other attributes using the spatial position channel.

8.1 Choropleth map

A choropleth map shows regions as area marks, and a quantitative attribute is encoded as color over regions; also, texture can be used as a channel. Usually employed for showing how a quantity is distributed across geographical areas.

It is a good choice for an overview rather than for an accurate comparison, because of the use of the color channel for quantitative data.

One of the main risks of this visualization is the confusion of geographic area and population with data values, e.g., showing where people live instead of actual data.

The major design choices for a choropleth map are: how to construct the color map and what region boundaries to use, i.e., spatial aggregation (should the region granularity be a country, a state, or a city?).

Overall, the choropleth map is useful to display how data is distributed over geographical regions. The ideal situation would be when regions have approximately the same size, so always ask yourself whether the data should be normalized or not. An important point is to pay attention to the choice of the color map according to the semantics of the data (is it sequential or diverging?) and to the number of bins. It can be an easy tool for conveying messages, thanks to the familiar and natural representation of the spatial data. However, be careful to not to pass the wrong message, and how the data is presented, it is easy to communicate the area (bin size) rather than the actual data, also the strong dependence on color might make it confusing.

Recap: Choropleth map

Data (What)	Geographic geometry data using a table with one quantitative attribute per region
Encode (How)	The use of given geometry for area mark boundaries to represent space, and the use of sequential segmented color map

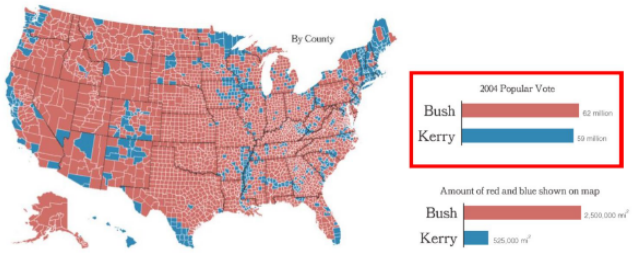


Figure 8.1: Choropleth map and message conveying ability

8.2 Symbol map

A symbol map, which can be a disk, a square, or a shape, is placed at a point and scaled such that its area is proportional to the quantity associated with the point. We can use area, shape, and color as channels. Unlike a choropleth map, a symbol map avoids confusing geographic areas with data values. One can even use graphs/charts/complex glyphs as symbols, even encoding multiple attributes.

A symbol map can be intuitive and easy to understand, they solve the issue of region dimension vs the value of the visualized attribute, because the symbol dimension is proportional to the data value and not to the region dimension. However, when encoding the data using a symbol map, one should try to avoid complex symbols, for they can be harder to understand; and the symbols' placement can be an issue: some symbols can overlap and hide other symbols or even region boundaries.

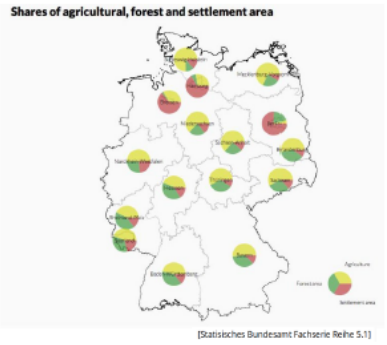


Figure 8.2: Symbol map

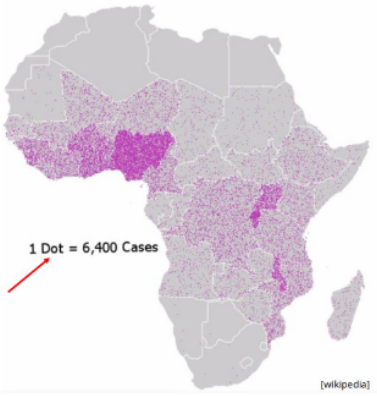


Figure 8.3: Dot density map

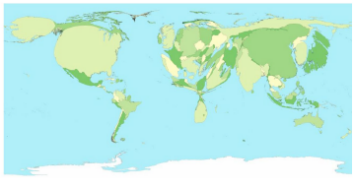
8.3 Dot density map

A dot density map typically uses points as marks, and points are positioned over a map, where each dot represents a given constant quantity of items; all dots are equal, it is their quantity and distribution that gives information.

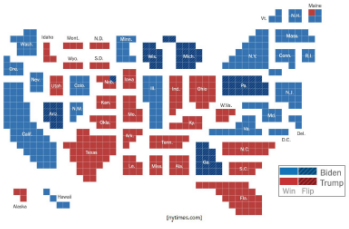
A dot density map is easy and intuitive, which mitigates the confusion between region areas and displayed values. But an accurate estimate can be difficult to make, due to the overlap of information and the quantity of dots. Sometimes, the normalization may be needed (for example, 1 dot = number cases / 1000 people).

8.4 Contiguous cartogram

It is a particular representation of the data where the geographic position and boundaries among regions are preserved, while dimensions are distorted according to a quantitative attribute. The aim is to preserve as much as possible the original position, shape, and boundary, and minimize distortion.



(a) Contiguous cartogram



(b) Grid variation

Figure 8.4: Contiguous cartogram and its grid variant

8.5 Best practice - ten rules

There are some recommended best practices that one should keep in mind when choosing a visualization tool. The ten rules are: best channel first; ensure accuracy and transparency; no unjustified 2D; no unjustified 3D; eyes beat memory; overview first, zoom and filter, details on demand; responsiveness is required; pay attention to color; function first, form next; self-contained visualizations

8.5.1 Best channels first

Recall the *effectiveness principle*: the relevance of information displayed should match the effectiveness of the channel. Meaning that the relevant information should be prioritized, then encoded with the most effective/accurate channels.

We have seen channel rankings; to follow this rule means to use the highest-ranked channels for the most important attributes, and the other channels for the remaining, secondary attributes.

8.5.2 Ensure accuracy and transparency

Recall the expressiveness principle: the visual representation should represent all of, and only, the relationships that exist in the data. This implies that information that is not present in the data should not be visualized. It is also important to not to convey misleading information while paying attention to axis starting values and scale, and align data when possible for accurate comparisons, and more.

8.5.3 No unjustified 3D

Misconception: *if two dimensions are good, then three dimensions must be better*.

There are many difficulties in visually encoding information with the third spatial dimension, depth. 3D visualization is justified only when the user's task involves shape understanding of inherently three-dimensional structures; usually, these inherently spatial data are scientific visualization and computer graphics, where the use of 3D is fundamentally required for understanding the three-dimensional geometric structure of objects or scenes. In all other contexts, the use of 3D needs to be carefully justified.

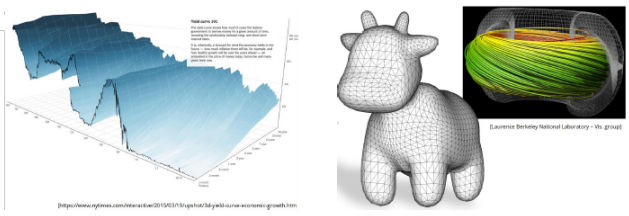


Figure 8.5: Justified 3D use cases

One of the main issues is occlusion, where some objects cannot be seen because they are hidden behind others. The overarching problem with occlusion in the context of visual encoding is that presumably important information is hidden (the occluded detail might even be critical), and discovering hidden details via navigation has a time cost.

In visually encoding abstract data, perspective is another very bad thing. Perspective distortion is one of the main dangers of depth because the power of the plane is lost; it completely interferes with visual encoding that uses the planar spatial position channels and the size channel, e.g., it is more difficult to judge and compare bar heights in a 3D bar chart than in multiple horizontally aligned 2D bar charts.

Another problem with the use of 3D is dramatically impaired text legibility. Text fonts have been very carefully designed for maximum legibility when rendered on the grid of pixels that makes up a 2D display. Text tilted in any way off the image plane typically becomes jaggy.

(a) Perspective issue

(b) Text legibility issue

Figure 8.6: Unjustified 3D use cases

8.5.4 No unjustified 2D

Laying out data in a 2D space should be explicitly justified, compared with the alternative of simply showing the data with a 1D list.

8.5.5 Eyes beat memory

Using our eyes to switch between different views that are visible simultaneously has a much lower cognitive load than consulting our memory to compare a current view with what was seen before.

The phenomenon of change blindness is that we fail to notice even quite drastic changes if our attention is directed elsewhere. Although we are very sensitive to changes at the focus of our attention, we are surprisingly blind to changes when our attention is not engaged. The difficulty of tracking complex and widespread changes across multi-frame animations is one of the implications of change blindness for visualization.

In simpler words, seeing lasts longer than remembering, but distraction prevents us from noticing changes.

8.5.6 Overview first, zoom and filter, details on demand

This is a visualization idiom that provides an overview is intended to give the user a broad awareness of the entire information space with the goal of summarizing. An overview helps the users to find regions where further investigation in more detail might be productive. An overview is often shown at the beginning of the exploration process to guide users in choosing where to drill down to inspect in more detail.

8.5.7 Responsiveness is required

The latency of interaction, namely, how much time it takes for the system to respond to the user action, matters immensely for interaction design. Human reaction to phenomena is best modeled in terms of a series of discrete categories, with a different time constant associated with each one. A system will fell responsive if these latency classes are taken into account by providing feedback to the user within the relevant time scale.

From a user's point of view, the latency of an interaction is the time between their actions and some feedback from the system indicating that the operation has completed. In a visualization system, that feedback would most naturally be some visual indication of state change within the system itself. The user should have some sort of confirmation that the action has completed. If an action could take significantly longer than a user would naturally expect, at least some kind of progress indicator should be shown to the user.

8.5.8 Pay attention to color

The use of color should be justified, because there are higher-ranked channels than color. Items should be easy to tell apart by using contrast. A limited number of available hues and a limited number of bins should be used (avoid similar colors). Take into consideration color blindness.

Hue for categorical attributes, saturation, and luminance for attributes for which ordering is meaningful.

Get it right in black and white

It is a design guideline for effective use of color, ensuring that the most crucial aspect of visual representation is legible even if the image is transformed from full color to black and white. This suggests encoding the most important attribute with the luminance channel to ensure adequate luminance contrast, and considering the hue and saturation channels as secondary sources of information.

8.5.9 Function first, form next

The best visual design should be both beautiful and effective; however, given an effective but ugly design, it is possible to refine the form to make it more pleasing while maintaining the base of effectiveness. In contrast, given a pleasing but ineffective design, you will probably need to toss it out and start from scratch since progressive refinement is not possible.

Visual beauty does matter, given that visualization makes use of human visual perception. Good visual form enhances the effectiveness of visual representations.

8.5.10 Self-contained visualizations

Look at the visualization, which should be enough to understand the visualization itself. The labels, titles, legends, and descriptions can be added if needed, but if more information is required for understanding the visualization, one should rethink the design.

9 Lecture 9 - Multidimensional data

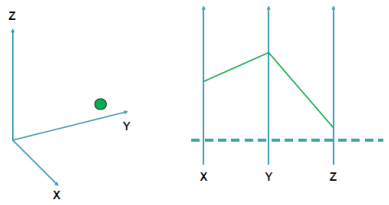


Figure 9.1: Parallel coordinates and cartesian coordinates

Parallel coordinates can be used to visualize multivariate data as an extension of Cartesian coordinates. Note that the order of the axes plays a fundamental role in its readability.

Together with parallel coordinates, scatterplot matrices can be used to represent multidimensional data (among other visualization tools). However, a different approach exists, which is to *reduce the dimensionality* of the data. This is a widely used strategy to map the dimensionality of the dataset from a higher N-dimension to 2 or 3 dimensions for a better understanding. Note that dimensionality reduction is a mapping, not a geometric transformation. It has different techniques, each with different properties.

9.1 PCA - Principal Component Analysis

PCA is a linear, non-parametric multidimensional reduction technique. The core idea is to find a basis formed by the directions that maximize the variance of the data, so that it can express the data in a new basis that best expresses our dataset.

Mathematically, if X is our original dataset and P is the transformation matrix, the new data Y is obtained as $Y = XP$. In this transformation, each row of $P(p_1, \ldots, p_m)$ represents a new direction (principal component), and Y represents the projection of the original data onto these directions

$$PX = \begin{bmatrix} \mathbf{p}_1 \\ \vdots \\ \mathbf{p}_m \end{bmatrix} [\mathbf{x}_1 \ \ldots \ \mathbf{x}_n] \tag{9.1}$$

Covariance Matrix

The role of the covariance matrix is to find the best directions. PCA looks at the covariance Matrix (C_X) of the original data such that $C_X = 1/(n-1)XX^T$, where the diagonal terms represent the variance of every single variable, while off-diagonal terms represent the covariance (redundancy) between different variables.

The goal is to find a matrix P such that the covariance matrix of the transformed data (C_Y is diagonalized, in other words, a matrix P that maximizes the variance and minimizes the redundancy.

- Maximize variance implies that high values on the diagonal mean the dynamics of the variables are captured effectively.
- Minimize redundancy implies that the low off-diagonal values mean variables are no longer redundant.

Solving PCA

Theorem 9.1. *A symmetric matrix A can be diagonalized by a matrix formed by its eigenvectors as $A = EDE^T$, where the columns of E are the eigenvectors of A.*

The solution relies on Theorem 9.1. The steps to compute PCA are:

1. Organize data as a $m \times n$ matrix
2. Subtract the mean from each row
3. Calculate the eigenvalues and eigenvectors of XX^T
4. The eigenvectors organized by their corresponding eigenvalues from the matrix P

Remember that $Y = PX$, we define the covariance matrix of the new data $C_Y = 1/(n-1)YY^T$

1. substitute Y with PX: $= 1/(n-1)(PX)(PX)^T$
2. resolve the equation: $= 1/(n-1)PXX^TP^T$
3. apply the theorem 9.1: $= 1/(n-1)P(XX^T)P^T$
4. as the result of Theorem 9.1, we obtain that $C_Y = 1/(n-1)PAP^T$, where P is transformation matrix and A is the symmetric matrix

Reducing dimensionality and reconstruction error

Reduction occurs when we choose only the first k principal components ($k < m$). By projecting the data onto these k directions, we get a lower-dimensional representation that is the best approximation of the original data.

PCA minimizes the reconstruction error e, defined as the sum of squared differences between the original points (y_i) and the projected points ($\tilde{y}_i$):

$$e = \sum_{i}^{k} (\tilde{y}_i - y_i)^2 \tag{9.2}$$

In fig. 9.2 shows a visual example of a ball attached to a spring moving along a single axis (the x-axis). If three different cameras record this movement, the raw data has 6 dimensions (x, y, coordinates for each of the 3 cameras). Because the ball only moves in 1D, most of these 6 dimensions are redundant or contain noise, but applying PCA allows the system to identify the single primary direction of motion, effectively reducing 6 noisy dimensions back to the 1 meaningful dimension that captures the most variance.

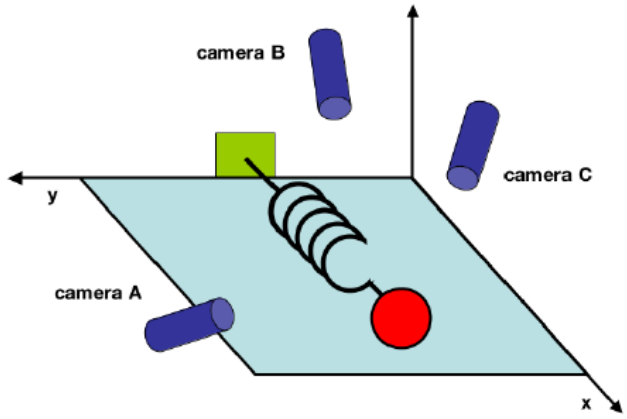


Figure 9.2: visual example of PCA dimensionality reduction

Limitations

- Non-parametric: while a strength, it can be a weakness because it does not incorporate prior knowledge
- distribution: it fails for data that is not Gaussian-distributed
- linearity: it is a linear technique; for non-linear structures, extensions like kernel PCA or other methods are needed
- independence: PCA does not guarantee statistical independence

9.2 MDS - Multi-Dimensional Scaling

MDS is a classic technique for dimensionality reduction. The primary rationale behind MDS is to find a lower-dimensional representation of data that preserves the pairwise distances between points as accurately as possible.

- The mapping: it seeks a linear mapping ($y_i = Mx_i$) that minimizes a cost function ($\phi(Y)$) representing the differences between the Euclidean distance of points in the high-dimensional space (d_{ij}^2) and their distance in the low-dimensional space ($||y_i - y_j||^2$)

$$\phi(Y) = \sum_{i,j} d_{ij}^2 - ||y_i - y_j||^2 \tag{9.3}$$

- relationship to PCA: minimizing the distance mismatch ($\phi(Y)$) is mathematically equivalent to maximizing the variance of the points in the new space, which is the exact goal of PCA.

MDS vs PCA

While the two techniques share similar goals and both better preserve large pairwise distances, they differ significantly in their computational scaling:

- PCA scales based on the dimensions of the data (the size of the covariance matrix is proportional to the number of variables)
- MDS scales based on the number of data points. This makes MDS a distinct choice depending on whether a dataset has many variables or many individual samples.

Sammon mapping variant

It is a specific adaptation of MDS that allows for retaining the local structure of the data better than classical MDS, where retention of high distances is privileged, and not of local structure. Sammon mapping addresses this by weighting the contribution of each pair of points.

9.3 LLE - Locally Linear Embedding

LLE is a dimension reduction technique designed to discover non-linear structures in high-dimensional data by exploiting local linear approximations. While PCA is a linear technique that looks at global variance. LLE operates on the intuition that even if a dataset is globally non-linear, it can be approximated as locally linear.

Core intuition

The fundamental assumption of LLE is that the high-dimensional data lies on a "well-sampled" manifold. Because the manifold is well-sampled, any given data point and its immediate neighbors can be treated as a small, locally linear patch. Each point in this patch can be represented as a weighted sum of its neighbors.

The LLE algorithm

LLE follows a three-step mathematical process to map high-dimensional points into a lower-dimensional space:

1. **identify neighbors**: for every data point X_i in the high-dimensional space, the algorithm finds its K-nearest neighbors or all points within a specific ball radius.
2. **compute reconstruction weights**: the algorithm calculates the weights W_{ij} that best reconstruct each point X_i from its neighbors by minimizing the error function:

$$\varepsilon(W) = \sum_i |X_i - \sum_j W_{ij} X_j|^2 \tag{9.4}$$

   In this step, the weights capture the local geometric properties of the data
3. **map to low-dimensional space**: finally, the algorithm finds new vectors Y_i in a lower-dimensional space (2D or 3D) that respect those same weights. It does this by minimizing a new cost function while keeping the weights W_{ij} fixed:

$$\Phi(Y) = \sum_i |Y_i - \sum_j W_{ij} Y_j|^2 \tag{9.5}$$

   This ensures that the local neighborhood relationships from the original high-dimensional space are preserved in the new, reduced space.

9.4 Isomap

The Isomap is a dimensionality reduction technique based in the core principle of preserving the geodesic distance between data points. It is designed to understand the intrinsic geometry of a curved space (a manifold).

In a high-dimensional space where data lies on a curved surface, the straight-line Euclidean distance between two points can be misleading because it "cuts through" the empty space of the manifold.

Definition 9.1 (Geodesic Distance)**.** *Geodesic distance is the shortest path between two points measured along the surface of the curved space.*

An isomap aims to "unroll" this curved manifold into a flat representation while keeping the distances along the surface consistent.

Procedure

The Isomap algorithm can be seen as a three-step mathematical procedure:

1. **construct neighborhood graph** (G): define a graph over all data points by connecting all points (i, j) if and only if point i is one of the K-nearest neighbors of point j

2. **compute the shortest path**: using Floyd's algorithm (or a similar shortest-path algorithm) to calculate the shortest path between all pairs of points in the graph *G*. This results in an approximation of the global geodesic distances between all points in the dataset.
3. **construct the d-dimensional embedding**: use the matrix of approximated geodesic distances to find a lower-dimensional representation.

Applications

Isomap is particularly effective for datasets where the "meaningful" dimensions are non-linear:

- Face poses: when applied to a dataset of faces, Isomap can successfully map images onto axes representing "up-down pose" and "left-right pose", capturing the natural rotation of a head
- Handwriting: Isomap can be used to visualize how different digits vary based on their "top arch" or "bottom loop" articulation

9.5 Autoencoder

An autoencoder is a specialized type of neural network used to perform dimensionality reduction. These networks offer a powerful way to map complex data into simpler forms. A multi-layer autoencoder consists of two primary components that work together to process data:

- The **encoder** that takes the high-dimensional **input data X**, and passes it through several layers of neurons. Each layer uses a set of weights to progressively compress the information.
- At the center of the network is a **"bottleneck" layer**. This layer holds the compressed, **low-dimensional representation (Y)** of the input data, effectively capturing its most essential features
- The **decoder** performs the reverse operation of the encoder by using a mirrored set of transposed weights to attempt to reconstruct the original data from the compressed representation. Thus, obtaining the final **output data X'**

The network is trained to ensure that the output data X' is as close to the input data X as possible. Because the data must pass through the low-dimensional bottleneck Y, the autoencoder is forced to learn a highly efficient way to represent the data.

To make it work for dimensionality reduction, once the network is trained, the decoder part can be discarded, and the encoder can be used as a tool to transform high-dimensional datasets into a 2D or 3D representation for visualization and analysis, similar to the goals of PCA or Isomap.

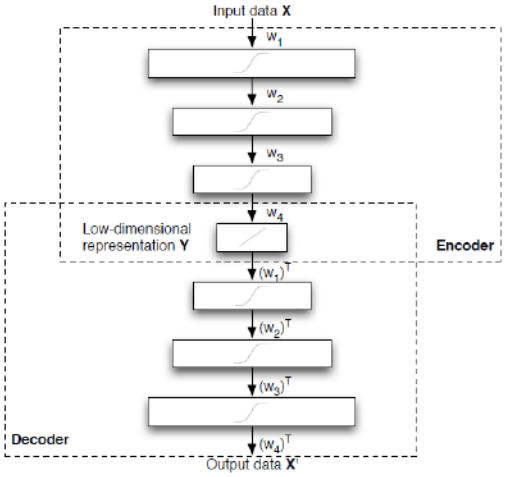


Figure 9.3: Multi-layer autoencoder

9.6 t-SNE

t-SNE stands for t-Distributed Stochastic Neighbor Embedding, a state-of-art dimensionality reduction technique to a common problem: the inability of most techniques to retain both local and global structure of data in a single 2D or 3D map.

9.6.1 SNE

Before introducing t-SNE, it is important to understand its predecessor. SNE models the relationships between data points using conditional probability rather than fixed distances.

- High-dimensional similarity ($p_{j|i}$): in the original high-dimensional space, the similarity of point x_j to point x_i is the probability that x_i would pick x_j as its neighbor. This is calculated using a Gaussian distribution centered at x_i
- Low-dimensional similarity ($q_{j|i}$): an analogous probability is calculated for the points in the lower-dimensional space

The objective is to ensure that the low-dimensional map Q reflects the high-dimensional neighborhood structure P as closely as possible, i.e., minimizes the mismatch between $p_{j|i}$ and $q_{j|i}$, this can be achieved using the KL divergence.

9.6.2 Kullback-Leibler (KL) Divergence

To quantify how much the low-dimensional map differs from the original data, SNE uses KL Divergence, also known as relative entropy.

- Coding theory view: it represents the expected number of extra bits required to code samples from distribution P if the code is optimized for distribution Q.
- Bayesian view: it measures the information gained when one revises beliefs from a prior distribution Q to a posterior distribution P.
- Mathematical property: a critical detail noted in the sources is that KL divergence is not symmetric, meaning $KL(P||Q) \neq KL(Q||P)$.

9.6.3 Into t-SNE

Standard SNE has two major flaws: its cost function is difficult to optimize, and it suffers from the "crowding problem", where points in the low-dimensional map tend to collapse together into a cluttered center. t-SNE solves these through two specific innovations:

- **Symmetry**: t-SNE makes the SNE cost function symmetric, which simplifies the optimization process $KL(P||Q) = KL(Q||P)$
- **Student-t distribution**: instead of the Gaussian distribution, t-SNE uses the Student-t distribution with one degree of freedom to evaluate similarities in the low-dimensional space. The "heavy tails" of the Student-t distribution allow points that are far apart in the high-dimensional space to be modeled as even farther in the low-dimensional space, which effectively alleviates the crowding problem.

10 Lecture 10 - Data Visualization in Python

Python is a perfect glue between several data manipulation tools, scientific libraries, and visualization toolkits. It is easy to learn and is a continuously supported language by the computer scientist community.

10.1 NumPy

NumPy is a specialized Python tool that Python has for efficient storage and manipulation of numerical arrays. NumPy arrays are like a built-in list type, but the tool supports efficient storage and data operations as the array grows larger in size.

A Python list is a data structure containing a reference count, a data type, the size in bytes, and the content. Hence, a Python list is a pointer to a block of pointers, which makes it flexible at the cost of efficiency.

On the other hand, a NumPy array is a fixed-type ndarray containing non-redundant information, which allows high efficiency in terms of memory and processing with reduced flexibility.

10.1.1 NumPy array manipulation

Array slicing

- accessing a single element: A[3, 2], denote the element at the 3rd row and 2nd column
- slicing: A[start, stop, step]
- first 5 elements: A[:5]
- elements from index 5: A[5:]
- sub-array: A[3:8]

Sub-arrays as no-copy views

Slicing operations on an ndarray return a view, not a copy of the array data. If we modify the sub-array, the corresponding part of the array would be updated. To create a copy of the sub-array, an explicit *copy()* method needs to be called.

Example:

–> x_sub = x[:2, :2].copy()

Reshaping

The *reshape()* method is used to change the array structure.

Example:

–> x.shape() returns (6, 3), a 6x3 matrix

–> y = x.reshape (18, 1)

–> y.shape() returns (18, 1), a 18 elements vector

Concatenation and Splitting

Concatenation() method concatenate two or more arrays or other methods such as *vstack()* for vertical stacking (rows) and *hstack()* for horizontal stacking (columns).

Analogously, an ndarray can be split using methods like *split()*, *hsplit()* and *vsplit()*

10.1.2 ndarray computations

Standard Python loops, such as the *for* loop, are extremely slow for elaborating a large quantity of numerical data. This is because Python is a runtime compiled language.

However, NumPy provides an interface for this kind of statically typed compiled runtime using vectorized operations.

Universal Functions or Ufuncs are the vectorized operations in NumPy, which execute element-wise operations with low latency. Some examples include: arithmetic operations (+, -, *, /); absolute values, trigonometric functions (sin, cos); exponents and logarithms; and specialized functions.

10.1.3 Broadcasting

Broadcasting is a set of rules that allow NumPy to execute operations between arrays and matrices of different sizes without creating redundant copies in memory, thus avoiding explicit loops and storage inefficiency. It has fundamentally three rules:

1. If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side
2. if the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape
3. if in any dimension the size disagrees and neither is equal to 1, an error is raised

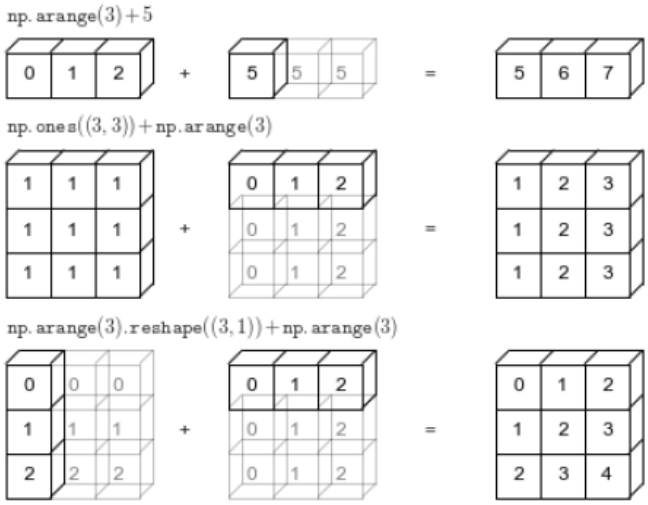


Figure 10.1: Broadcasting rules explained

10.2 Pandas

Pandas is an open-source library that provides high-performance, easy-to-use data structures and analysis tools. Its primary objects, *Series and DataFrames*, are built on top of NumPy arrays to provide both the flexibility needed for data wrangling (like handling labeled data) and the efficiency of numerical arrays.

10.2.1 Series

A Series is defined as a one-dimensional array of indexed data. While a NumPy array defines an implicit integer index, a Pandas Series allows for an explicitly defined index. A Series can also be thought of as a specialized type of dictionary where keys are the indices and values are the data points.

Examples:

–> data = pd.Series ([a, b, c, d], index=[1, 2, 5, 7])

–> data[5] returns *c*

10.2.2 DataFrame

While a Series is 1D, a DataFrame is a two-dimensional array with flexible row indices and flexible column names. It can be viewed as a sequence of aligned Series objects that share the same index.

10.2.3 Indexing and Selection

Pandas can sometimes be ambiguous when using integer indices

–> Is data[5] refers to the explicit label or the implicit fifth position?

To solve this, the library provided specific indexer attributes:

- **loc**: always refers to the **explicit** index (the user-defined label)
- **iloc**: always refers to the **implicit** index (the Python-style positional index starting from 0)
- **ix**: a hybrid of the two, though less commonly used in the modern versions.

Examples:

–> data = pd.Series ([a, b, c, d], index=[1, 2, 5, 7])

–> data.loc[1] returns *a*

–> data.iloc[1] returns *b*

–> data.loc [1:2] returns [1, a], [2, b]

–> data.iloc[1, 2] returns [2, b], [5, c]

One can access the column name in a dictionary-style (–> data['population']) or in an attribute-style (–> data.population). However, the assignment works only for a dictionary-style.

Indexing conventions for DataFrame

While indexing refers to columns, slicing refers to rows. Also direct masking operations are interpreted row-wise rather than column-wise.

–> data[data.density > 100] returns rows of the dataframe that satisfies the condition

Ufuncs preserve index and column labels in the output, and for binary operations like addition and multiplication, Pandas automatically align indices.

10.2.4 Handling missing data

Pandas is particularly robust at managing "junk" or missing data. It uses *None* for generic object data and *NaN* for numerical values. It can automatically convert between these types when appropriate (if inserting None into an integer array will convert the array to floating-point and use NaN).

It offers built-in methods to work on missing data, either to fill in, to remove, or to replace. For example, *dropna()*, which can remove missing values from a Series or entire rows/columns from a DataFrame.

10.3 Data visualization tools in Python

Python offers a vast and interconnected landscape of visualization libraries. At the heart is Matplotlib, which serves as the foundation for many other popular tools like Seaborn and the built-in plotting functions in Pandas. Other libraries include JavaScript-based tools like Plotly and Bokeh, and OpenGL-based tools for high-performance rendering.

Matplotlib

This library is heavily inspired by Matlab, making it very easy for researchers moving from that environment to adapt.

- **Pros**: It is extremely flexible and supports many different rendering backends
- **Cons**: The API is imperative (you need to tell it exactly how to draw every piece), the default style is considered poor, and it lacks interactivity; it can be slow when handling very large datasets or complex designs

Matplotlib inspired tools

- **Pandas**: Its goal is to provide a simple way to visualize a DataFrame directly, often including more complex graphs with less code than raw Matplotlib
- **Seaborn**: This library is specifically targeted toward statistical data visualization. It provides much aesthetically pleasing default "nice style" and simplifies the creation of complex plots, such as joint plots that combine scatter plots with histograms or kernel density estimates.

Plotly

It is a modern alternative to the Matplotlib family, built on JavaScript. It provided high interactivity, allowing users to zoom, hover for data, and toggle categories. While it uses JavaScript for the rendering, a version called Dahs is available that allows Python users to build interactive dashboards without needing to write any JavaScript code.

Imperative vs Declarative

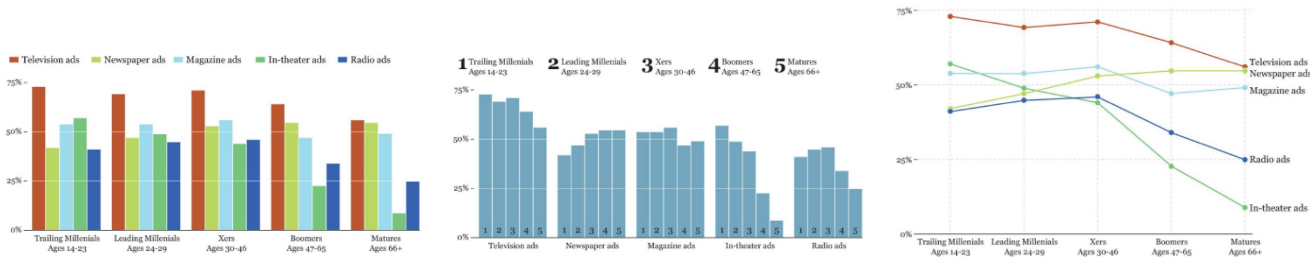
Imperative means that you have to manually specify the graphs and the related details.

Declarative means that you have to specify what should be done with the data, and the framework decides the details.

11 Lecture 11 - Data manipulation and Visualization

Visualization is a tool to enable specific tasks or convey particular messages. One should choose a graphic form based on what you want to achieve: comparing values, seeing changes, revealing relationships, or envisioning patterns.

To organize a display effectively, one should identify the specific tasks the graphic should enable, try different graphic forms according to tasks, arrange the graphic components to make it easy to extract meaning, and test the outcome with other people.



(a) Grouped bar chart *by age* (b) Grouped bar chart *by media* (c) Line chart

Figure 11.1: Advertisement market analysis example

In fig 11.1 shows an example of advertisement market analysis to show the degree of advertising media influence on each age group in buying decisions. It shows how reorganizing the same data can reveal different insights:

- Fig.11.1a groups all media types under each age category. It is excellent for comparing different methods within a single age group.
- Fig.11.1b organizes the data so that each media type is its own category, with age groups as the bars within them. This makes it easier to see a downward pattern as the population ages.
- Fig.11.1c places age groups on the x-axis and media types as separate lines; this chart allows for a dual purpose. It shows trends across age groups while also allowing the comparison between each medium within each group because the data points are "stacked" visually.

11.1 Exploring data

The core objective of data exploration is to find patterns and trends in data and then observe deviations from those patterns, often referred to as identifying the **"norm (or smooth) and the exception"**.

When beginning to explore a dataset, it is suggested to use a measure of central tendency, which gives an idea of the "average" size of numbers and where the center points lie.

Mode, Median, Mean The **mode** is the simplest measure and represents the values that appear most frequently in the distribution. Unimodal distributions have one mode, multimodal distributions have two or more modes that are equally common.

The **median** is the value that divides the distribution exactly in two halves; it is considered a resistant statistic because it is not distorted by extreme outliers.

The **mean** or average is the result of adding up all values and dividing by the total count. This measure is very sensitive to extreme values (outliers), unlike the median.

Example: The raw data of annual salary, in $ [20k, 22k, 25k, 30k, 32k, 40k, 5M]

–> the mean annual salary of this dataset is around 700k$, which is a dozen times higher than the majority of the annual salaries in the dataset

–> the median of the data is 30k$ which is a more accurate representation of the average

Weighted means

It is important to be careful when calculating simple statistics and understanding the data before manipulating it.

When calculating the average of several already-averaged groups, it is calculating a **grand mean** (or "mean of means"). This approach is appropriate if all the groups have a similar number of counts. If the count sizes are different, a discrepancy will emerge; the risk is that this approach gives a small group exactly the same weight as a large group.

Example: School A has 5 students while School B has 500 students. If the grand mean is used here, a single high-performing student in the tiny school A will have a disproportionately massive impact on the final average, while the performance of School B is undervalued.

To account for these size differences, the **weighted mean** needs to be used, which ensures that each individual data point contributes equally to the final result, regardless of which group they belong to.

Range and Shape

While measures of central tendency tell you where the "center" of your data is, they are often insufficient on their own. They do not reveal whether the data points are clustered closely around that center or widely scattered.

The simplest way to measure spread is the range, which is the difference between the maximum and the minimum values in a distribution. The range tells you the extremes, but it does not tell you how many data points are close to the average.

Histograms can be used to visualize frequency and the **shape** of the dataset. Data is aggregated into "bins" (ranges of scores), and the height of each bar represents the frequency of values within that range.

If a histogram is too aggregated, a strip plot can provide a higher level of detail by displaying every single data point as a dot.

Takeaway message

The histogram, the range chart(or lollipop chart), and the strip plot are three different ways to visualize the spread and the shape of the distribution at different levels of detail. Using them together allows one to see the "norm" while identifying the "exceptions". One should never rely on just a single statistic or chart for exploratory work.

11.2 Numerical summaries and Standardization

Transforming visual representations of spreads to more precise numerical summaries and techniques for standardizing data allows for fair comparisons between different groups, providing exact measurements of how much the data deviates from the "norm".

The two most common tools for measuring the "spread" of a distribution are *variance* and *standard deviation*, which are defined as follows:

$$variance = \frac{\sum(Each\ score - mean)^2}{\#scores} \tag{11.1}$$

$$standard\_deviation = \sqrt{variance} \tag{11.2}$$

When you need to compare individual scores from two completely different distributions, you use a **z-score** (or standard score). A z-score tells you exactly how many standard deviations a raw score is away from its mean. It is defined as:

$$z = \frac{(raw\_score - mean)}{standard\_deviation} \tag{11.3}$$

Example: IT salary comparison in the US (high mean, low relative standard deviation) Vs Nigeria (low mean, high relative standard deviation).

–> Because the cost of living and economies are different, one cannot compare directly absolute values.

–> By transforming salaries into a standard score, one can find the "equivalencies". For instance, if an employee in the US has a z-score of 0.3, you can calculate what a similar relative salary would be in Nigeria by applying that same 0.3 z-score to the Nigerian distribution.

–> Given that in the US an employee makes 125.335, and that the US mean is 122.400, the standard deviation is 10.746. Then the z-score of this employee = (125335-122400)/10746 = 0.3

–> Find the "equivalent" salary in Nigeria by finding the raw score, given that in Nigeria, the standard deviation is 12.589 and the mean is 29170. The equivalent salary in Nigeria is (0.3 * 12589) + 29170 = 32947

The standard Normal distribution

Many natural phenomena (like height, weight, exam scores) follow a normal distribution, in which:

- the mean, median, and mode are all the same
- the distribution is perfectly symmetrical
- **empirical rule**: approximately 68.2% of cases fall within 1 standard deviation of the mean, 95.4% within 2 standard deviations, and 99.8% within 3 standard deviations.

The empirical rule explains how "normal" or "exceptional" a specific data point is: if a value has a z-score of 1, you know it is higher than roughly 84% of the population (the 50% below the mean plus the 34.1% between the mean and 1 standard deviation). If a value falls beyond 2 standard deviations, it is statistically rare.

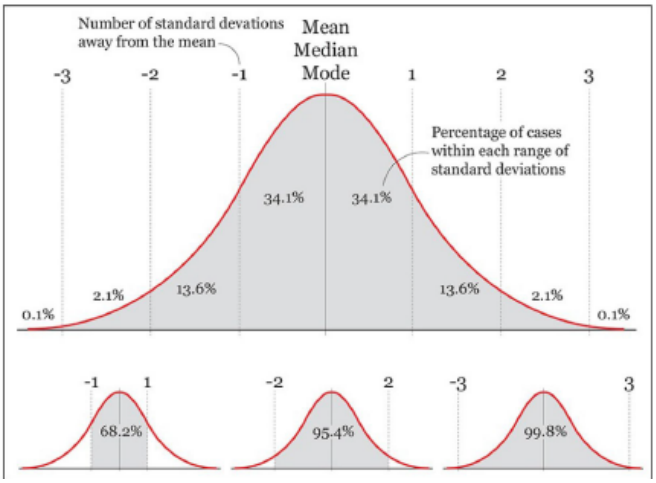


Figure 11.2: Standard Normal Distribution

Standard scores can be misleading

The population size matters!! Given the example in the reading material (page 45), the graphics show that sparse, rural counties in the US has both the highest and the lowest cancer rates. This is not

necessarily due to diet or environment; it is a statistical reality that small populations show much more variation and more extreme outliers.

One way to visualize this data properly is to use a funnel plot, where the variation of data points narrows significantly as the population size (on the x-axis) increases.

11.3 Percentiles

Measurements like *median* and *percentiles* are resistant statistics; they are not distorted by extreme outliers in a distribution.

Definition 11.1 (Percentile)**.** *A percentile p is a value that splits a distribution such that p% of all other values lie below it.*

For instance, the median is always the 50th percentile because it lies exactly in the middle. These act as a rank; being in the 90th percentile means your score is equal or higher than 90% of the rest of the group. Quartiles (the 25th, 50th, and 75th percentiles) divide the distribution into quarters, and are recommended when exploring data.

11.3.1 Box plot

The box plot is a simplified visual representation of a distribution that emphasizes quartiles and outliers. Its main components are:

- the **box**: represents the inter-quartile range (IQR), which is the distance between the first and the third quartiles. The vertical line inside the box marks the median
- the **whiskers**: these lines extend from the box up to 1.5 times the IQR on either side
- **outliers**: any value that falls beyond the whiskers is considered an outlier and is plotted as individual dots.

While less detailed than a histogram or a strip plot, box plots are superior for certain tasks:

- **stressing outliers**: they make extreme values stand out much more clearly than other char types
- **comparing distributions**: they are excellent for placing multiple datasets side-by-side to compare their centers and spread at a glance
- **revealing exceptions**: by "setting aside the level" (subtracting the mean of a group from each individual score), a box plot can be used to visualize only the departures from the norm, helping one spot the most unusual cases in a dataset

11.4 sum up

Exploring data consists of observing trends and patterns (the norm) and then identifying deviations and exceptions from them.

The norm is made of three main features: the level (measures of central tendency), the spread, and the shape of the data.

Exploring exceptions often involves transforming the data to set aside one of the elements of the norm, i.e., transforming the original datasets into standard scores, setting aside the spread, and substituting it with a standardized one, based on the number of standard deviations from the mean.

11.5 Time series

The standard tool for visualizing change over time is the line chart, where the x-axis represents equally spaced time intervals, while the y-axis corresponds to the magnitude of the variables being explored.

When analyzing these charts of time series data, it is important to keep an eye out for the *trend* (the overall direction over the segment being explored), the *seasonality* (consistent, periodic fluctuations that can often muddle the understanding of the underlying trend), and the *noise* (random variations that do not follow a specific pattern).

When looking at the data, raw numbers can be misleading; to get a better understanding, one should apply several transformations:

- **contextual comparisons**: you might compare current data to a yearly average or look much further "back in time" to see if a recent change is truly unusual.
- **ratios vs. raw numbers**: for example, when looking at the social security enrollment, the raw number of people might be increasing, but if the total population is growing faster, the percentage of people enrolled might actually be decreasing.
- **moving average**: to create a better model for noisy data, you can use a moving average. This involves dividing the time series into chunks (e.g., 6-month periods) and calculating the average of each, which smooths out short-term fluctuations.
- **plotting residuals**: once you have a moving average, you can plot the "departure" or residuals (the raw score - moving average) to see exactly how and when the data deviates from the trend.

Logarithmic transformations

When dealing with variables that grow very quickly, such as population growth or bacterial cultures, standard linear scales often fail to show early changes clearly. The problem lies in an exponential growth pattern; the changes on a standard line chart only become noticeable very late in the process. Therefore, by applying a log10 transformation to change the scale so that each unit increase represents

a tenfold increase. This results in a much easier identifiable growth rate, which usually appears as a straight line.

12 Lecture 12 - Graph drawing

Definition 12.1 (graph drawing)**.** *Graph drawing is the process of taking an abstract graph $G = (V, E)$, consisting of a set of vertices (V) and a set of edges (E), and using an algorithm to produce a visual "drawing".*

Graph drawing is essential across many scientific and technical fields where relationships between entities need to be visualized. For instance, in engineering, it is used for designing electric circuits; in computer science, it helps in website analysis and visualizing computer networks; in social and security science, the emerging applications include intelligent analysis, forensic accounting, and more.

Graph drawing is closely linked to several other fields: graph theory (mathematical properties of graphs), information visualization (using visuals to explore abstract data), computational geometry (solving geometric problems using computers), and algorithm engineering (devising efficient computational processes).

12.1 Planarity

Definition 12.2 (Planar embedding)**.** *Planar embedding is a way to map the graph to a 2D plane such that no edges intersect*

The primary challenge in graph drawing is to find a planar embedding. To simplify the problem, it is suggested to break the graph down based on its connectivity:

- cut-vertex: a specific vertex whose removal produces a disconnected graph
- bi-connected graph: a graph that contains no cut-vertices
- blocks: you can recursively split a graph into smaller bi-connected pieces, which are called blocks
- Block-Cut-vertex Tree (BC-tree): the relationships between these blocks and the cut-vertices used to split them are represented by a specialized tree structure

Theorem 12.1 (Planarity)**.** *A graph is planar if and only if all of its individual blocks are planar*

Thanks to this theorem, which allows researchers to use a divide-and-conquer strategy: instead of testing a massive, complex graph all at once, you can test each block separately.

12.1.1 The Auslander and Parter algorithm

It is an established algorithm with computational cost of $O(n^2)$, a method for planarity testing.

1. **Bi-connectivity assumption** –> the algorithm assumes the input graph is bi-connected. If the graph is not bi-connected, the divide-and-conquer strategy can be used to test its individual blocks one by one.
2. **Selecting a cycle and identifying "pieces"** –> the process begins by finding a cycle C within the graph, and the algorithm identifies the "pieces" attached to C. A **piece** can be a single edge connecting two non-consecutive vertices on the cycle, or a more complex subgraph connected to the cycle at specific points. Every piece must eventually be drawn either entirely inside or entirely outside the cycle C to avoid crossing edges
3. **Building the incompatibility Graph** –> the algorithm builds a secondary graph, called the incompatibility graph, to determine if a piece can be placed without crossing. Each node in such a graph represents one of the pieces identified in the previous step. An edge is drawn between two nodes if their corresponding pieces conflict. A conflict occurs if the two pieces cannot be placed on the same side of the cycle without crossing each other
4. **Bipartite-ness** –> once the incompatibility graph is built, the algorithm checks if it is bipartite. A graph is **bipartite** if the pieces can be partitioned into two groups, those that are drawn inside the cycle and those that are drawn outside. If it is not bipartite, it means that the graph contains a cycle of conflict that makes it impossible to draw it without at least one crossing. In this case, the graph is non-planar
5. **Divide-and-Conquer recursion** –> passing the incompatibility test for each cycle is not enough. The algorithm must ensure that the internal structure of each piece is also planar. The algorithm repeats the entire process recursively for each piece, combined with the cycle it is attached to. The recursion stops when a piece is reduced to a simple path or a single edge, at which point it is inherently planar.

12.2 Graph drawing aesthetics

While an algorithm can produce many different drawings of the same graph, not all of them are equally useful. Aesthetics can be seen as mathematical properties that correlate with the readability of a drawing.

Minimizing crossing

The most fundamental aesthetic is reducing the number of edge intersections. A drawing is clearer if the number of intersections is reduced. This is why planarity testing is so critical. A planar drawing (no crossings) is generally the gold standard for readability.

Minimizing size (area)

A good drawing should be compact. Small size allows the user to see a large picture of the drawing at once. One should try to minimize the total area of the drawing, the maximum edge length, and the average edge length.

Minimizing bends

In certain drawing conventions (like "orthogonal" drawings where edges are made of horizontal and vertical segments), edges can have "bends" or angles. Edges with many bends are difficult to follow for human eye.

The contrast of aesthetics

A crucial insight is that these aesthetics often conflict with each other. You cannot always minimize everything at once. Choosing the "best" drawing often involves a trade-off between these competing goals

drawing conventions

When an algorithm is tasked with creating a drawing, it must follow a drawing convention, a set of geometric rules that define the style and constraints of the output.

- Polyline: edges are represented as a sequence of connected line segments
- Straight line: edges are simple, single straight segments between vertices
- Layered: vertices are arranged into horizontal layers, often used for hierarchical structures
- Orthogonal: edges are made up only of horizontal and vertical segments

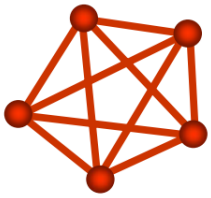
Drawing on the grid can ensure precision and help minimize the total area of the drawing. In a grid drawing, vertices are required to have integer coordinates. This approach helps satisfy the aesthetic goal of small size, which allows users to see more of the graph at once.

12.3 Kuratowski's theorem

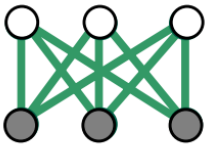
While many graphs can be drawn planarly, others are inherently non-planar. The Kuratowski theorem provides criteria for identifying these graphs

The minimal non-planar graphs

The two minimal non-planar graphs: there are two specific structures that make a graph non-planar. Fig 12.1a is a complete graph with 5 vertices where every vertex is connected to every other vertex. Fig



(a)



(b)

Figure 12.1: Non-planar drawings

12.1b is a complete bipartite graph where two sets of three vertices are connected to each other, also known as the "utility graph"

The subdivision rule

A graph does not have to be exactly one of the two drawings in fig. 12.1 to be non-planar; it only needs to contain a subdivision of one of them. A subdivision occurs when new vertices are inserted along the edges of the original structure.

13 Lecture 13 - Visualizing large graphs

The primary non-technical challenge in visualizing large datasets can be defined as a perceptual scalability problem. While the technical challenge is the computational memory issue. When a dataset contains too many items or too many attributes, the resulting visualization becomes an *information overflow*.

13.1 Overview and Details

The core solution to the problem, allowing a user to analyze data that is too large for their display, is to *combine representation and interaction*. This can be achieved by balancing two types of views: overview and details.

- **Overview** provides a high-level view of the entire dataset. It is valuable for representing overall patterns, assisting with navigation, and orienting the user's activities. It can be one of the two paradigms: *perspective* (a panoramic big picture from which trends and categories can be inferred) or *compendium* (a sketchy view where high-level categories and trends are shown directly)
- **Details** allow the viewer to examine individual cases and attributes, provided on demand, meaning only a few are shown at a time to prevent clutter

To allow a user to navigate a dataset that is too large for their screen, the designer must decide how to display the overview (or context) and the detail (or focus) together. There are two primary ways to combine these views: space-based combination and time-based combination.

Space-based combination

This approach uses different portions of the screen to show the overview and details at the same time. These are often called *Focus + Context techniques*, and they are classified into several categories:

- **Details-only (single window)**: this is the simplest version, where interaction happens strictly through zooming and panning. It works effectively when the zoom factor is relatively small; otherwise, the "far away" context can be easily neglected.
- **Coordinated pair**: this provides a combined display of the overview and a local magnified view. It can be implemented as two separate views side-by-side or by replacing the overview as an inset within the detailed view (like mini-maps in Google Maps)

- **Thumbnails**: these provide a summary overview of a document, such as the slide panel used in PowerPoint
- **Fisheye view**: inspired by physical lenses, this technique magnifies the center of the field of view while providing a continuous fall-off in magnification towards the edges. This allows the user to see local details and global context simultaneously without switching windows. This may have a single or multiple focuses. An example of this technique is the toolbar in a Mac; the icon enlarges when the mouse passes over it
- **Perspective wall**: this technique uses a 3D interface to fold a large data space, allowing a much larger area to be seen within a small window
- **Space folding**: this involves "folding" a visual representation (like a map) to eliminate non-relevant parts of the data, effectively bringing the focus and context closer together.

Time-based combination

This approach involves alternating between the overview and the details sequentially in the same space. Users navigate the data using temporal interactions like zooming, panning, and scrolling rather than seeing both views at once.

13.2 Interactive exploration strategies

Users navigate and explore massive datasets through specific interactive strategies. These solutions rely on a strong interaction between the user and the drawing, where the graph is visualized incrementally or at varying levels of detail.

There are two fundamental, opposing strategies for exploring large graphs: *Top-down and Bottom-up*

13.2.1 Top-Down exploration (The Shneiderman Mantra)

This is the most common approach and follows the Shneiderman mantra: **overview first, zoom and filter, then details on demand**.

The approach works starting at an initial "sketchy" or high-level overview of the information. The user then interactively explores the data by filtering out noise or zooming in on specific areas to get more detail.

The high-level overviews are typically created using three techniques: **filtering** (deleting nodes/edges), **sampling** (keeping only a fraction), or **clustering** (merging nodes).

However, this approach has a major limitation: the definition of what constitutes a "good" overview is highly dependent on the specific problem or dataset at hand

13.2.2 Bottom-Up Exploration

This strategy is used primarily when an overview is impossible to get because the graph is simply too vast or complex. Its mantra is: **search, show context, expand on demand**.

This approach works by visualizing the graph one piece at a time. The users navigate through the data using a "topological window" that acts like a lens moving across a canvas, or through the incremental enhancement of a specific starting point.

But, because there is no global overview, it is very difficult for users to preserve a mental map of where they are in the larger structure of the graph.

13.3 Overview generation

This section covers the techniques used to generate the upon mentioned overviews. These techniques focus on reducing the complexity of a graph while preserving its most important structural features.

13.3.1 Filtering and Sampling

Filtering involves deleting nodes and edges to simplify the drawing. There are three common criteria for deciding which nodes to keep:

- Node degree: this assumes that nodes with many connections (high degree) are more important than those with few connections
- Node role: this focuses on "bottleneck" nodes that keep different parts of the graph connected; these are prioritized over redundant nodes
- Random choice: every data point has the same probability of being selected, which is useful for getting a statistically representative "sample" of a massive graph

13.3.2 Coreness and k-core

A more sophisticated form of filtering is based on coreness. Instead of simply deleting low-degree nodes once, the algorithm performs a recursive deletion.

Definition 13.1 (k-core)**.** *The k-core of a graph is the maximal induced subgraph such that every remaining node has degree of at least k.*

The k-core of a graph is obtained by recursively deleting all nodes of degree less than k. Because deleting a node reduces the degree of its neighbors, the algorithm continues to recursively remove these newly degree-reduced nodes until every node left in the subgraph satisfies the degree requirement.

The algorithm creates a hierarchy of k-cores: 1-core is obtained by removing all isolated nodes; 2-core is obtained by recursively removing all nodes of degree 1; 3-core is obtained by recursively

removing nodes of degree 1 and 2, and so on, until the algorithm reaches a point where no nodes meet the degree requirement (an empty graph, so to speak).

Note: the subgraphs of a k-core do not have to be connected, therefore, *subgraphs* :)

Properties

If, for some k >= 1, a graph G has k-core G_k, it has following properties:

- **Uniqueness**: for any given value of k, the k-core of G_k is unique
- **Existence**: if a graph has a k-core (G_k), it must also possess a (k-1)-core (G_{k-1})
- **Nesting**: the k-core is always contained within the (k-1)-core ($G_k \subseteq G_{k-1}$), forming a clear structural hierarchy.

Higher k-cores represent the increasingly "dense heart" of a network.

Application

The k-core components are the individual connected pieces that make up the k-core. These components are often used in the *(X, Y)-clustering model*, where the goal is to group nodes such that the "graph of cluster" (X) is sparse and readable (like a planar graph), while each "individual cluster" (Y) is a k-core component.

Determining the minimum k for which a graph can be visualized as a (planar, k-core component)-graph can be solved in polynomial time, typically $O((n+m)\log_n)$, making it a highly efficient strategy for navigating complex network data.

13.3.3 Betweenness centrality

Another metric used for filtering is betweenness, which is a measure of centrality used to identify the relative importance of nodes within a network. Betweenness focuses on a node's role as a bridge or connector along the shortest paths between other nodes. Nodes with high betweenness are critical for the flow of information, even if they don't have a high degree.

Definition 13.2 (Betweenness)**.** *Let $\delta_{st}(v)$ denote the fraction of the shortest paths between two nodes, s and t, that pass through vertex v such that, $\delta_{st}(v) = \sigma_{st}(v)/\sigma_{st}$, where:*

- *$\sigma_{st}(v)$ is the # of the shortest paths from s to t passing through v*
- *σ_{st} is the total # of shortest paths from s to t*

The betweenness *of a vertex v is the sum of all fractions of every pair of nodes in the graph passing through vertex v.*

Role

In the context of managing "information overflow" in large datasets, betweenness is a critical metric for filtering and sampling.

- Nodes with high betweenness are often "bottlenecks" or "bridges" that keep different parts of the graph connected.
- When creating a high-level overview, these "bridge" nodes are considered more important than redundant nodes, even if they have a relatively low number of direct connections (degree)

By retaining nodes with high betweenness while filtering out others, designers can create a high-level overview that preserves the essential topological structure of a massive graph.

13.4 Clustering and clustered graphs

Clustering is a primary method for creating high-level overviews of large graphs by merging individual nodes into groups. Each cluster acts as a single representative for all the nodes it contains, effectively reducing the visual complexity of the network.

13.4.1 Hierarchical and Flat clustering

There are two main structural organizations for clusters, hierarchical (or recursive) clustering and flat clustering:

- **Hierarchical clustering** is where clusters can be nested within one another, creating an inclusion tree. This structure allows users to explore complex information at different abstract levels by expanding or contracting specific branches of the tree
- **Flat clustering** occurs when all clusters (other than the root) are direct children of the root. This is functionally equivalent to a coloring of the vertices in the underlying graph, where each color represents a distinct group

Definition 13.3 (proper inclusion tree)**.** *The inclusion tree (hierarchical clustering) is* proper *if all leaves are at the same distance from the root*

Definition 13.4 (view)**.** *A* view *of a clustered graph is the graph obtained by only considering the clusters of a specific level of a proper inclusion tree*

13.4.2 Visualizing clustered graphs

There are several conventions for rendering these structures:

- 2D node-link diagrams: these use standard representations for the network, but add regions bounded by simple curves to represent clusters. These regions must contain all and only the nodes belong to that cluster, and their inclusion within other regions represents the hierarchy
- 3D multilevel drawing: these display different levels of this inclusion tree as separate layers, using "inclusion" edges to link the hierarchy between levels
- clustered matrices: this approach represents the graph as a matrix where rows and columns can be recursively contracted and expanded to show different levels of detail
- surface visualizations: clusters can be represented as surfaces where the exact position of nodes is known and fixed, but details only appear when the user focuses on that specific cluster.

The clusters can have different origins: *manual* (explicitly defined by the users), *extrinsic* (based on external meta-information on classification data), or *intrinsic* (derived solely from the structure of the graph, such as through property-based or cut-based classification).

13.4.3 Clustered Planarity (C-planarity)

While standard planarity only requires that edges do not cross, a C-planar drawing must satisfy stricter geometric constraints to avoid information overflow. Specifically, you must avoid three types of intersections:

- edge-edge crossing
- edge-region crossing, in which an edge cannot cross the boundary of a cluster region unless one of its endpoints is inside that cluster
- region-region crossing, in which the boundaries of two different cluster regions cannot intersect.

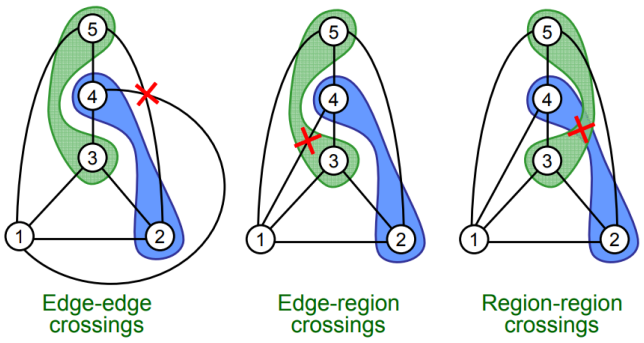


Figure 13.1: C-planar crossings

The difficulty of testing whether a clustered graph can be drawn without these crossing depends on its connectivity. These graphs were the subgraph induced by every single cluster that is connected. As for the complexity, the planarity problem is polynomial. But if the clusters are not connected, the complexity remains unknown.

13.4.4 Clustering quality

After establishing the planarity rules, we also need to assess the quality of the chosen clustering for the task visualization. This can be evaluated via two metrics: coverage and performance.

Coverage is the ratio of intra-cluster edges (edges within a group) to the total number of edges in the graph, while *performance* is a count of "correctly interpreted pairs", which includes edges within clusters plus pairs of nodes in different clusters that are not connected.

K-way partitioning

K-way partitioning is a technique which attempt to divide a network into k clusters while minimizing inter-cluster edges (bridges between groups). This is noted as an NP-hard problem.

Alternatively, recursively partitioning the graph into two cluster until some ending condition is reached.

13.4.5 (X, Y)-clustering model

(X, Y)-clustering model is a formal framework for navigating large graphs by balancing high-level readability with low-level detail. **Class X** (the "norm") represents the graph of clusters; to guarantee readability, class X should be a class of sparse graphs, such as planar graphs, trees, or cycles. **Class Y** (the "exception") represents the internal structure of each cluster; class Y should be a class of highly connected graphs, such as cliques or k-core components.

While determining if a graph fits a (planar, k-clique) model is NP-hard, identifying a (planar, k-core component) graph can be solved in polynomial time $(O(n+m)\log\ n)$. This is achieved by computing the core numbers of all vertices and performing a binary search to find the minimum k that allows for a planar overview.

14 Lecture 14 - Rendering

Definition 14.1 (Rendering)**.** Rendering *is the process of converting input data, such as 3D geometry and material properties, into a final image. To visualize a 3D scene, it must be transformed into a synthetic raster image that can be displayed on a screen.*

To render a scene effectively, an algorithm requires several types of input: *geometric data* (the raw shapes of the objects), *material and texture* (information about how surfaces look and react to light), a *synthetic scene* (the arrangement of these objects in a virtual space), *camera model* (a mathematical description of the "picture-making apparatus").

The material categorizes rendering into three families of algorithms:

- *Ray tracing*: tracing the path of light to simulate visual effects
- *Path tracing*: a specialized, more advanced form of ray tracing
- *Rasterization-based pipeline*: the standard approach for real-time graphics, which converts 3D primitives directly into 2D pixels

The Pinhole camera

The pinhole camera is a fundamental mathematical model used in computer graphics as a metaphor for the virtual camera, describing the relationship between an observer and a 3D scene.

It is modeled as a parallelepiped (a box) where the front face contains an infinitesimally small hole. When the light enters through this hole, the resulting image is formed on the back side of the box, known as the focal plane.

This pinhole camera is formed of components: the *viewpoint* is the eye position or the projection center located at the pinhole itself; the *focal distance* (d) is the distance between the pinhole (projection center) and the back side of the box where the image forms; while a physical pinhole camera forms an inverted image on the back of the box, computer graphics conventions often assume the existence of an image plane located between the scene and the projection center to simplify calculations.

The model allows for precise control over what is visible in the rendered scene. The angle of view θ can be controlled by varying the ratio between the focal distance d and the height of the image plane h.

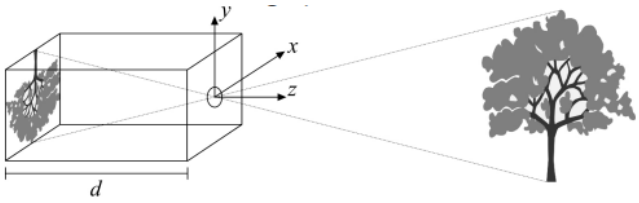


Figure 14.1: Pinhole camera

14.1 The chain of transformations

Definition 14.2 (chain of transformations)**.** *The chain of transformations is the technical process of transforming 3D models into 2D synthetic scenes through a sequence of mathematical mappings.*

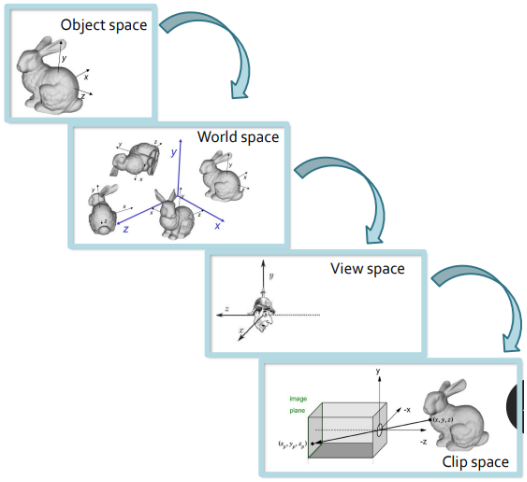


Figure 14.2: The chain of transformations

This chain of transformations requires the definition and computation of proper transformations. The hierarchy of space is the following:

- **Object space** (local space): this is the reference frame where a 3D model is originally defined. Each object has its own space, arbitrarily chosen by the modeler to define its specific point coordinates and normals
- **World space**: because a scene contains many different models, a common reference system is needed to arrange them; this is the world space. The **modeling transformation** maps an object from its local space into this shared world space, defining its position, rotation, and scale within the scene
- **View space** (camera/eye space): this space has its origin at the camera center. A second transformation moves objects from the world space into the view space so that their coordinates are relative to where the "eye" sees

- **Clip space**: this is the final destination, aligned with the 2D screen of the output device. Mapping to this space involves projecting 3D elements onto a 2D plane

14.1.1 Affine transformations

Definition 14.3 (Affine transformations)**.** *In Euclidean geometry, an affine geometry is a geometric transformation that preserves lines and parallelism.*

We need transformations to translate, scale, rotate, and deform 3D objects, to go from the object to the world space. These transformations are called affine.

The affine transformations send points to points, lines to lines, planes to planes, and so on, while *maintaining the ratios of distances* between points on a straight line (midpoint of a segment goes to midpoint of a segment). However, affine transformations do not necessarily preserve angles between lines or distances between points. Examples of affine transformations include translation, rotation, scaling, shear, reflection, and compositions of them in any sequence.

14.1.2 Composing transformations

Often, multiple transformations (like rotating an object and then translating it) need to be combined. The problem is that while rotation and scaling are expressed as products (multiplication), translation is expressed as a sum, making them difficult to combine into a single operation.

A possible solution is to use the so-called homogeneous coordinates, that is, a different representation for points and vectors that adds an extra dimension with respect to Cartesian coordinates (a third coordinate w for 2D, or a fourth for 3D).

Homogeneous coordinates

Given a 2D point P(x, y) is represented as (x_h, y_h, w), with $w \neq 0$. To convert back to original Cartesian coordinates, you divide by the extra dimension: $x = x_h/w$ *and* $y = y_h/w$. By convention, w is usually set to 1 for specific locations (x, y, 1).

A big advantage in using this system is that, all affine transformations, including translation, can be represented as 3×3 matrices (for 2D) or 4×4 matrices (for 3D). This facilitates the mathematical concatenation of several transformations into one by simply multiplying the matrices. This results in a single "master matrix" that can be applied to every point in a 3D model in one step.

We can also use homogeneous coordinates to remove points/vectors ambiguity, by setting w coordinates to 1, we have points (locations); and when w = 1, we have vectors (directions) representing "points at infinity".

14.2 Ray tracing

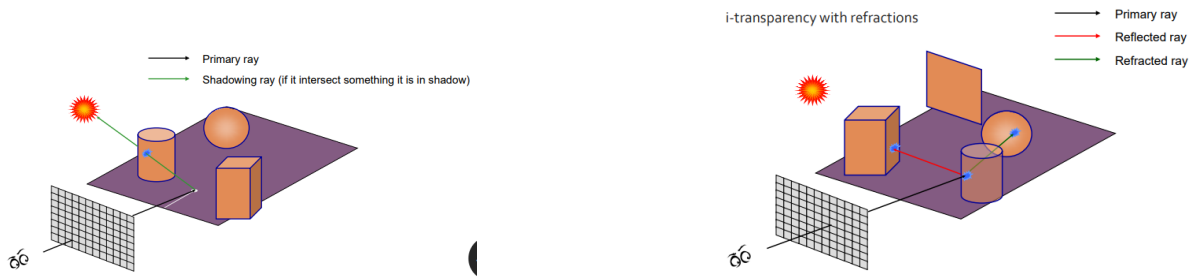
Definition 14.4. *Ray tracing is a rendering paradigm that simulates the way light behaves in the real world by following its path.*

The main idea of the ray tracing is tracing backwards. In nature, light rays travel from a source, bounce off objects, and eventually enters our eyes. Because the vast majority of light rays in a scene never reach the observer, simulating all of them would be a vast amount of energy. So, instead of tracing light from the source, ray tracing works backward. For each pixel on the screen, the algorithm shoots a ray, called a **primary ray**, from the viewpoint into the scene. And the algorithm finds where that ray intersects with the closest primitive (object) in the 3D scene. The color of that pixel is then determined by the properties of the object hit at that specific point.

Secondary rays

One of the primary advantages of ray tracing is that it handles complex optical phenomena "naturally" through the use of secondary rays:

- To see if a point is in shadow, the algorithm shoots a "**shadowing ray**" from the intersection point toward the light source. If it hits an object on the way, the point is in shadow.
- If the ray hits a mirror-like surface, a new "**reflection ray**" is bounced off the surface to see what other objects are reflected in it.
- For semi-transparent objects like glass or water, the algorithm calculates a "**refracted ray**" that passes through the material, allowing for a realistic simulation of transparency.



(a) Shadowing ray

(b) Reflection and refraction rays

Figure 14.3: Ray tracing: primary and secondary rays

Primitives and geometry

Ray tracing offers more freedom in the types of shades it can render compared to other methods. While most real-time systems rely on triangles, ray tracing can intersect rays with any mathematical

primitive that has a defined intersection formula, including spheres, implicit surfaces, and Geometric Solid Modeling (GSM).

Computational cost

Historically, ray tracing was considered "too slow" for real-time use because the core operation, computing the intersection between a 3D ray and 3D primitives, is complex. However, the cost is actually sublinear relative to the number of objects in the scene, provided the right data structures are used. And to speed up calculations, spatial indexing is used, a specialized hierarchical or hashing-based data structures that allow the algorithm to quickly skip parts of the scene the ray won't hit.

Modern GPUs, like NVIDIA RTX, now combine ray tracing with traditional methods to achieve the effects in real-time.

14.3 Rasterization

Rasterization is the alternative to ray tracing, which is the standard method for real-time graphics. While ray tracing works by shooting rays from the observer for each pixel, rasterization takes the opposite approach: **it takes each 3D object and determines which pixels it covers on the screen**.

Definition 14.5 (Rasterization)**.** *Rasterization is a rendering paradigm that converts 3D geometric data into a 2D raster image by determining which pixels on the screen are covered by each 3D object.*

Because modern GPUs are highly optimized for rasterization, almost every 3D scene is decomposed entirely into triangles before being rendered. Rasterization is also frequently called the **T & L pipeline**, consisting of two primary computational goals:

- **Transform**: the algorithm projects 3D primitives (geometric shapes) from their world coordinates into the 2D "screen space" of the output device
- **Lighting**: the algorithm computes the final color for every part of the scene, taking into account the geometry, the light sources, and the optical characteristics of the material

Rasterization process

The process flows through four distinct stages to turn a vertex into a pixel:

1. Per-vertex transformations: individual vertices and their attributes are moved into the correct coordinate spaces
2. Primitive processing: the vertices are assembled into basic shapes, while points and lines are possible, triangles are the universal standard because graphics HW is specifically optimized to process them

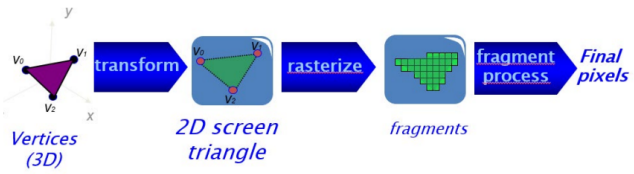


Figure 14.4: Rasterization-based rendering

3. Rasterization: the core step where a 2D shape on the screen is filled with a grid of fragments. A fragment is essentially a "potential pixel" that carries all the data necessary to compute a final color.
4. Per-fragment computation: the final color of each fragment is determined, often by accessing texture RAM to apply images to the surface, before the results are written to the frame buffer.

Attributes Interpolation and Perspective correction

During rasterization, the values defined at the corners of the triangle, the vertices, must be spread across the entire surface. If *Barycentric coordinates* are used, the algorithm performs a linear interpolation of attributes like colors or normals across the face of a triangle. If *Perspective correction* is used, a simple linear interpolation of texture coordinates across a triangle can cause visual distortion due to the way perspective works. Rasterization HW must apply a perspective correction formula using the depth (w) coordinate to ensure textures appear correctly aligned as they recede into the distance.

Advantages and limitation

Rasterization remains the dominant paradigm despite the rise of ray tracing, thanks to the ease of **parallelism**, making it perfectly suited for the architecture of modern GPUs; the high **scalability** of image resolution, meaning increasing the number of pixels on a screen has a more predictable performance cost than in ray tracing.

However, a historical disadvantage of rasterization is that it cannot naturally handle "global" effects like shadows or reflections. But designers have developed "clever" advanced algorithms to approximate these effects with compromise-level quality in real-time.

14.4 Ray tracing Vs Rasterization

The debate between ray tracing and rasterization stems from their fundamentally different approaches to generating an image from 3D data. It is a choice between processing the scene "pixel by pixel" Vs "primitive by primitive".

Quick recap, in ray tracing, for each pixel on the screen, the algorithm checks every geometric

primitive in the screen to see if it is hit by a ray; in rasterization, for each geometric primitive (usually a triangle), the algorithm determines which pixels on the screen it covers.

Advantages of *Ray tracing*

Ray tracing is often preferred for high-fidelity, "off-line" rendering, such as in movies by Pixar or DreamWorks, for several reasons:

- **Global effects** –> it naturally handles complex optical phenomena like shadows, specular reflections, and refractions
- **Conceptual simplicity** –> the underlying algorithm is straightforward, you simply trace the path of light
- **Primitive freedom** –> it is easier to calculate the intersection of a ray with complex mathematical shapes (spheres, implicit surfaces) than it is to project and fill those shapes as pixels
- **Scene scalability** –> if a specialized data structure for "spatial indexing" is used, the computational cost can be sublinear relative to the number of objects in the scene

Advantages of *Rasterization*

Rasterization is the standard for "real-time" rendering, such as video games and web previews, because of its efficiency:

- **Parallelism and HW optimization** –> the algorithm is easily parallelizable and was the first to receive specialized HW support, GPUs
- **Resolution scalability** –> it scaled better with image resolution (the number of pixels) than ray tracing
- **Handling dynamic data** –> it is generally easier to manage scenes where objects are constantly moving
- **Speed** –> it is inherently fast because it only processes the specific pixels covered by a primitive rather than checking every primitive for every pixel

The reality of the debate...

The traditional boundaries between these two methods are blurring. The current state is that *rasterization is fast, but needs cleverness to support complex visual effects; ray tracing supports complex visual effects, but needs cleverness to be fast*.

A clever rasterization utilizes real-time engines that use algorithms and approximations to simulate the reflections and shadows that ray tracing produces naturally. While clever ray tracing uses efficient spatial queries to avoid being slow.

Modern technology, such as NVIDIA RTX, has effectively ended the binary debate by combining both rasterization and ray tracing within the same HW to achieve high-quality visual effects at real-time speeds.

15 Lecture 15 - Lighting

Lighting can be perceived and mathematically modeled in different ways:

- *Ray optics*: the light is modeled using rays. The rays follow geometric rules, which can model several lighting effects like reflection and refraction
- *Wave optics*: the light is modeled as a propagating wave, which can model lighting effects such as diffraction
- *Electromagnetic optics*: it can describe lighting effects like the polarization of the light, not explained by the wave optics
- *Photon optics*: quantum mechanics is used to model lighting effects

The lighting is a process where energy is emitted from a source (like a lamp or the sun), travels through a scene, and interacts with objects until it stabilizes. This happened at the speed of light, making the result appear instantaneous and table to the human eye.

When a ray of light hits a surface, several effects happen simultaneously: specular reflection (light bounced off in a specific direction, like a mirror), diffusion (light scatters in many directions), transmission (light passes through the material, refraction), absorption (the material soaks up the energy), and scattering/emission (effects like fluorescence).

Solid angle

To calculate lighting in 3D space, the solid angle (Ω) is introduced, which is the 3D extension of a 2D planar angle. It represents the angular dimension of a cone pointing in a specific direction. It is measured in steradians (sr). A full hemisphere corresponds to a solid angle of 2π steradians.

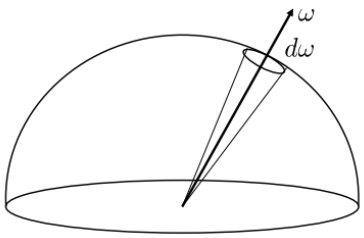


Figure 15.1: Solid angle

Radiometry

Radiometry defines three critical units used to quantify light in a scene:

- Radiant flux Φ, measured in Watts, this is the total radiant energy passing through a surface per unit of time
- Irradiance E, the radiant flux incident on a specific surface element ($Watts/m^2$)
- Radiance L, the radiant flux per solid angle per unit area ($Watts/m^2sr$). This is effectively what a camera or an eye "sees" from a specific direction

A fundamental takeaway is that for a point light source emitting light uniformly in all directions, *the intensity (irradiance) reduces with the square of distance* ($1/r^2$).

15.1 Rendering equation

Definition 15.1 (Rendering equation)**.** *The rendering equation is the fundamental formula for calculating the total visible light (L_o) at any point on a surface from a specific direction.*

The rendering equations calculate the visible light in a point of the scene for a specific direction, given by the sum of emitted light and the light reflected in that direction: *emitted light* (L_e), which is the energy the object produces itself; and *reflected light* (L_r), which is the light from the environment that bounces off the surface. To find the reflected light, the algorithm must integrate all incoming light contributions from every direction in the hemisphere above the point, weighting them according to the material's properties and the angle of reflection.

There are two specialized functions to mathematically represent how different surfaces (wood, metal, skin ..) look; these two are BSSRDF and BRDF

BSSRDF

Bidirectional Scattering Surface Reflectance Distribution Function.

The function describes the most general case of light-material interaction, where light hits a surface and may leave from a different position; the process is called scattering.

BRDF

Bidirectional Reflectance Distribution Function.

Because of the complexity of BSSRDF, computer graphics often uses the BRDF as an approximation. It assumes light enters and leaves at the same point x. It is defined as the ratio between the outgoing radiance (reflected light) in a certain direction and the incident irradiance (incoming light) from another direction.

15.2 Local lighting model

Because calculating the full rendering equation is computationally too expensive for real-time systems, designers use local lighting models to approximate the way light interacts with surfaces. These models focus only on the local effects, ignoring the complex "global" inter-reflection between all objects in a scene.

Definition 15.2 (Global effects)**.** *Global effects involve interactions between multiple objects, such as soft shadow, indirect lighting (color bleeding), and caustics*

Definition 15.3 (Local effects)**.** *Local lighting models simplify the scene by calculating light for a single point based only on its own properties and the direct light sources, ignoring reflections from other objects.*

15.2.1 Phong lighting model

The Phong model is a fundamental tool for simulating how *opaque* materials look under a point light source. While it is quite realistic, the sources note that some materials can end up looking like plastic. It simplifies the math by breaking the final color into three distinct components:

- **Ambient component**: this term is used to *roughly approximate global inter-reflections*. It provides a constant base level of light to all points in the scene so that areas not directly hit by a light source are not completely black
- **Diffuse components**: based on Lambert's law, this represents *light that hits a rough surface* (like wood or stone) and reflects uniformly in all directions. The intensity depends on the angle (θ) between the light source and the surface normal; it is brightest when the light is directly overhead
- **Specular component**: this models the "shiny" part of an object, the specular highlight. It *simulates light reflection off smooth, mirror-like surfaces* in a specific direction. Unlike diffuse light, the appearance of the specular highlight depends on the position of the observer.

A critical part of the specular term is the exponent (n). This parameter defines how "sharp" or "focused" the specular highlight is. A high value of n creates a very small, bright portion of light (like a polished billiard ball), while a lower value makes the highlight larger and more spread out.

The model also accounts for light attenuation, where the intensity of the light source decreases as the distance (d_L) from the light to the surface increases.

15.3 Shading

Definition 15.4. *Shading is the process of computing the actual color of each pixel on the screen based on the lighting models. It determines where and how often the light-material interactions are calculated*

on a 3D model

There are three primary types of shading: Flat shading, Gouraud shading, and Phong shading.

Flat shading

Flat shading is the simplest form of shading, where the lighting equation is computed only once for each face (polygon) of the 3D model. In which the algorithm uses the face normal to determine a single color, which is then applied uniformly across the entire surface of that face.

Because each face has a single solid color, the edges between them become very obvious, and the object looks "blocky." Even if you use a massive number of faces to try to smooth the object, it will fail to smooth the edges.

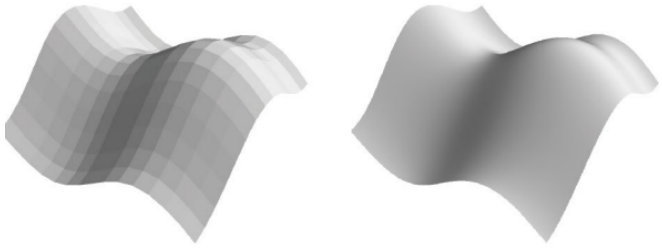


Figure 15.2: On the left, the flat shading, on the right, the reference object

The visual effect of **Mach Banding** is the perceptual phenomenon where the human eye exaggerates the contrast at the borders between different shades, making discontinuities clearly visible, thus explaining the presence of "blocky" edges.

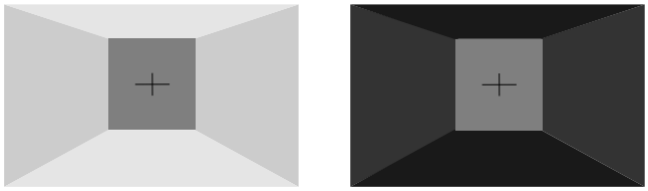


Figure 15.3: Mach Banding

15.3.1 Gouraud shading

To fix the blockiness of flat shading, Gouraud shading calculates the lighting at the vertices instead of the faces. The lighting equation is computed for each vertex using a vertex normal (often calculated as the average of the normals of the faces meeting at that vertex). The colors at the vertices are then linearly interpolated across the face of the triangle during rasterization. This creates a much smoother appearance, making faceted models look like continuous curved surfaces.

But, it struggles with specular highlights. If a highlight (the bright, shiny spot) falls in the middle of a face rather than near the vertex, it might be missed entirely or appear distorted because the interpolation only knows about the colors at the corners. It can also have visual issues at the corners of objects.

15.3.2 Phong shading

Phong shading provides the highest quality results by moving the lighting calculation from the vertices to the individual pixels (fragments). Instead of interpolating the colors from the vertices, this method interpolates the surface normals across the face of the triangle. The lighting equation is then computed for every single pixel using these interpolated normals.

This is the most realistic method out of the three. It correctly renders specular highlights even if they are very small or fall in the center of a face, and it provides a perfectly smooth appearance for curved surfaces.

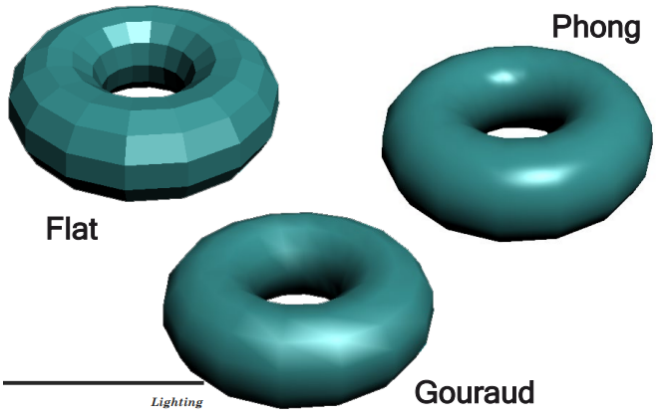


Figure 15.4: Shading methods result comparison

16 Lecture 16 - Texture

Texturing is a fundamental technology in computer graphics used to add high levels of visual detail to 3D surfaces without requiring complex geometric modeling.

Definition 16.1. *Texturing is the process of modulating the attributes of vertices, such as color, normals, or transparency, to obtain a specific visual effect.*

During rendering, textures are stored in a specialized memory on the graphics card called Texture RAM. Each texture is a 1D, 2D, or 3D array of **texels** of the same data type (a texel is the base element, just like a picture is made of pixels). Depending on what the texel represents, a texture can be a color map (RGB), an alpha map (transparency), a normal map (surface orientation: X, Y, Z), or a shininess map (specular coefficients).

The most common form of texturing is **texture mapping**, where the color attribute is modulated to wrap a 2D image around a 3D object.

To tell the computer how to wrap the image, every vertex in a 3D model is assigned coordinates in a 2D "texture space", also known as **uv space**.

16.0.1 UV space

Every vertex in a 3D mesh is assigned a coordinate (x, v) ranging from 0.0 to 1.0, which corresponds to a specific location on the 2D texture image. During rasterization, these u, v coordinates are interpolated across the face of a triangle so the system knows which part of the image belongs to which pixel.

These u, v coordinates can be assigned manually during modeling, precomputed, or generated automatically using projection functions. Different techniques of texture coordinates assignment need to be uses to speed up the texture transfer CPU RAM - Texture RAM. The specific technique to be used depends on the application.

Attribute interpolation using Barycentric coordinates and perspective correction is also used here in texturing.

16.0.2 Texture look up

When a pixel on the screen does not perfectly align with a texel in the image, the system must perform a texture look-up using filtering techniques:

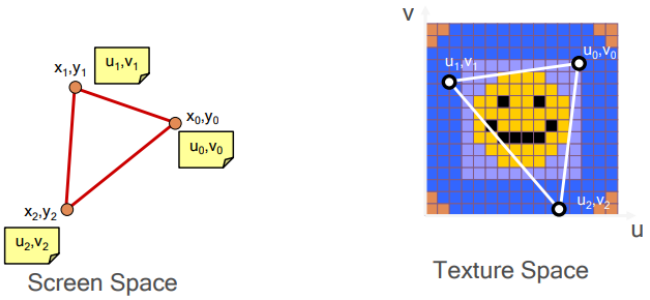


Figure 16.1: UV Space

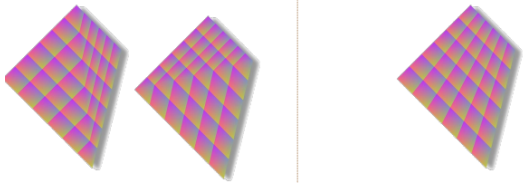


Figure 16.2: Left without interpolation, right with interpolation

- **Magnification** is used when a pixel is larger than a texel. To avoid a "blocky" look, the system uses *nearest filtering* (choosing the closest texel) or *bilinear interpolation* (averaging the closest texels for a smoother look)
- **Minification** is used when a pixel is smaller than a texel. This often causes visual artifacts like aliasing. The standard solution is *mip-mapping*, where the system pre-computes several lower-resolution versions of the texture and chooses the one that best matches the current viewing distance.

Figure 16.3: Magnification methods comparison

If a vertex is assigned texture coordinates outside the standard range, the system uses one of two modes:

- **Clamp mode**, any coordinate less than 0 is treated as 0, and anything greater than 1 is treated as 1, effectively stretching the edge texels
- **Repeat mode**, the integer part of the coordinate is ignored, causing the texture to tile across the surface, which is highly efficient for larger surfaces like brick walls.

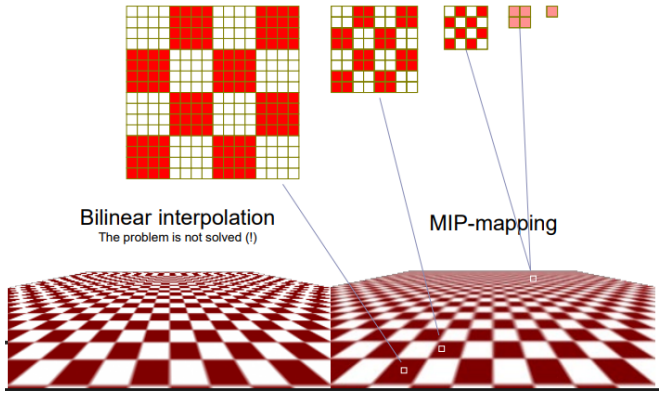


Figure 16.4: Mip-mapping visualized

16.0.3 Advanced mapping techniques

Building on the standard color modulation of texture mapping, the following advanced mapping techniques designed to simulate complex optical effects and intricate surface details.

Environmental mapping

The environmental mapping is used to simulate an object reflecting its surroundings without the high computational cost of full ray tracing. A texture, known as an environment map, captures the colors of the reflected scene. There are two primary methods for applying this:

- **Sphere mapping**: the technique maps the environment onto a virtual sphere surrounding the object. It uses spherical coordinates derived from the reflection vector to determine which part of the environment is visible from the observer's viewpoint.
- **Cube mapping**: this is often preferred because it provides more uniform sampling than sphere mapping and is easier to generate. The environment is projected onto the six faces of a virtual cube.

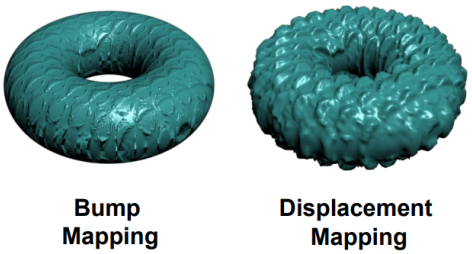
Bump mapping

Bump mapping is introduced as a way to simulate surface irregularities like wrinkles or bumps. Bump mapping alters the way light hits a surface without modifying its actual 3D geometry. The technique uses a texture (a normal map) to perturb the surface normals of an object. Because lighting calculations depend heavily on surface normals, the light "tricks" the eye into seeing depth and texture on a perfectly flat surface. However, because the geometry is unchanged, the silhouette of the object remains perfectly smooth.

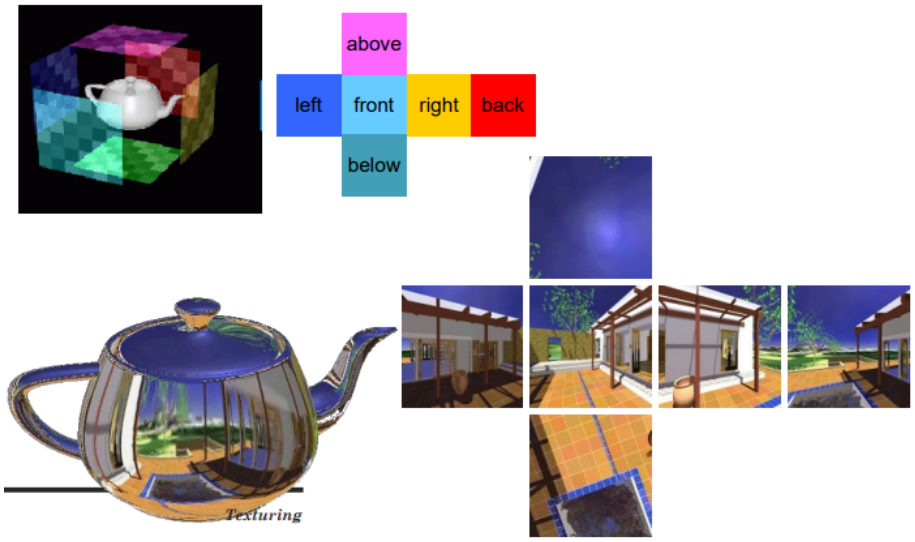
Displacement mapping

Displacement mapping is a more advanced alternative to bump mapping that provides a much higher level of realism. Unlike bump mapping, this technique effectively moves the 3D points of the surface during rendering. Because the geometry itself is moved, the silhouette of the model is correctly deformed. This allows for the realistic rendering of very rough surfaces, such as jagged rocks or heavy cratering, that bump mapping cannot convincingly simulate.

(a) Sphere mapping example



(b) Bump mapping and displacement mapping examples



(c) Cube mapping example

Figure 16.5: Texture techniques